

**A Final Dissertation Report on
Minimalist Alert Network for Threat Intelligence System**



Anuskar Sigdel

UON ID: 24812606

BSc. (Hons.) Computer Science



Supervisor: Nischal Khadka

Naaya Aayam Multi-Disciplinary Institute

University of Northampton

August 2026

Abstract

MANTIS, the Minimalist Alert Network for Threat Intelligence System, is a host based intrusion detection prototype for commodity Linux written entirely with the Python Standard Library. The project addresses a practical gap: small Linux servers, Internet of Things gateways and teaching laboratory machines need network monitoring, yet established platforms assume resources and dependencies that such hosts cannot spare, while opaque learned models resist operator explanation. MANTIS captures IPv4, TCP and UDP traffic through AF_PACKET sockets or offline PCAP replay and routes derived observations to five transparent engines: Z-Score statistics for volumetric floods, K-Nearest Neighbours for port scans, Multinomial Naive Bayes for malicious plaintext payloads, Mini-Batch K-Means for behavioural novelty and a Bloom Filter for known malicious sources. Alerts are enriched with MITRE ATT&CK context, persisted to CSV, forwarded as UDP JSON telemetry to a browser dashboard and may trigger policy gated iptables blocking. Learned state is exported to and restored from a JSON brain file. The artefact was developed with a Kanban process and evaluated through 116 automated unit, integration, system and regression tests, all passing, together with small domain matched checks reporting precision, recall, F1 and the Matthews correlation coefficient. Results are presented as laboratory verification within stated Linux, plaintext and resource limits rather than as production accuracy claims.

Acknowledgements

I would like to thank my supervisor, Nischal Khadka, for his guidance, patience and detailed feedback throughout every stage of this project. I am grateful to the staff of Naaya Aayam Multi-Disciplinary Institute and the University of Northampton for providing the environment and resources that made this work possible. Finally, I thank my family and friends for their constant support and encouragement during the preparation of this dissertation.

Table of Contents

Abstract	3
Acknowledgements	4
List of Figures	7
List of Tables.....	9
Abbreviations	10
1. Introduction	1
1.1 Background.....	1
1.2 Rationale.....	2
1.3 Aim and Objectives.....	3
1.4 Overview of Methodology	4
1.5 Limitations.....	5
2. Literature Review	6
2.1 Review of Related Research.....	6
2.2 Comparable Systems.....	9
2.3 Research Methodology.....	13
2.4 Implementation Traceability	14
2.5 Development Methodology.....	14
2.6 Evaluation Strategy	15
2.7 Methodology for Comparable Analysis	16
2.8 Requirement Specification	17
2.9 Algorithms Implemented.....	30
3. System Analysis and Design.....	37
3.1 Use Case.....	37

3.2 Sequence Diagrams.....	42
3.3 Class Diagram.....	45
3.4 Activity Diagrams.....	47
3.5 Database Schema	51
3.6 Algorithms Implemented.....	52
3.7 System Interface and Design	56
4. System Build and Technical Notes.....	66
5. Testing Strategy and Evaluation.....	68
5.1 Testing Definitions and Approach.....	68
5.2 Algorithm Tests	69
5.3 Test Cases and Scenarios	70
6. Future Scope.....	83
7. Conclusion.....	84
8. References	85
Appendix A: Gantt Chart.....	87
Appendix B: Supervisor Meeting Forms	88

List of Figures

<i>Figure 2.1: Snort product and documentation page (Cisco Talos, 2024).</i>	10
<i>Figure 2.2: Suricata documentation page (OISF, 2024).</i>	11
<i>Figure 2.3: Zeek documentation page (Zeek Project, 2026)</i>	12
<i>Figure 2.4: Wazuh documentation page (Wazuh, 2026)</i>	13
<i>Figure 2.5: Kanban development workflow from the project plan.</i>	15
<i>Figure 2.6: Qualitative comparable system functionality ranking using the project rubric.</i>	16
<i>Figure 2.7: Current monitoring and manual response workflow.</i>	27
<i>Figure 2.8: Proposed end to end MANTIS workflow.</i>	28
<i>Figure 2.9: Z-Score statistical engine flowchart.</i>	32
<i>Figure 2.10: K-Nearest Neighbours engine flowchart.</i>	34
<i>Figure 2.11: Multinomial Naive Bayes engine flowchart.</i>	36
<i>Figure 3.1a: MANTIS abstract goal level use case model.</i>	38
<i>Figure 3.1b: MANTIS level 1 use case model with include and extend detail.</i>	38
<i>Figure 3.2a: Live detection sequence from captured frame to operator visible alert.</i>	42
<i>Figure 3.2b: Training pipeline from validated data to persistent engine state.</i>	43
<i>Figure 3.2c: SIEM alert delivery and browser dashboard retrieval.</i>	44
<i>Figure 3.2d: Offline PCAP replay and finite analysis completion.</i>	45
<i>Figure 3.3: Corrected MANTIS class model with inheritance, composition and explicit UML multiplicities.</i>	46
<i>Figure 3.4a: Live detection, verdict aggregation and optional response activity.</i>	47
<i>Figure 3.4b: Training pipeline from input validation to brain export.</i>	48
<i>Figure 3.4c: Evaluation harness and confusion matrix metric activity.</i>	49
<i>Figure 3.4d: Corrected brain persistence and orderly shutdown activity flow.</i>	50

<i>Figure 3.4e: Linux installation and capability configuration activity.</i>	<i>51</i>
<i>Figure 3.5: Online Mini-Batch K-Means flow from explicit feature input to normal or anomaly output.</i>	<i>54</i>
<i>Figure 3.6: Bloom Filter reputation membership decision flow.</i>	<i>56</i>
<i>Figure 3.7a: Overview page wireframe showing system health, recent events and engine activity.</i>	<i>58</i>
<i>Figure 3.7b: Packets page wireframe showing capture controls, packet history and selected frame detail.</i>	<i>59</i>
<i>Figure 3.7c: Alerts page wireframe showing the complete bounded threat activity log.</i>	<i>60</i>
<i>Figure 3.7d: Architecture page wireframe showing all five engines and the response path.</i>	<i>61</i>
<i>Figure 3.8a: Overview page mockup with controlled health metrics, recent events and five engine activity.</i>	<i>62</i>
<i>Figure 3.8b: Packets page mockup with filters, protocol counters, exports and packet detail.</i>	<i>63</i>
<i>Figure 3.8c: Alerts page mockup with the complete in memory threat log and ATT&CK context.</i>	<i>64</i>
<i>Figure 3.8d: Architecture page mockup documenting all five engines and policy gated response.</i>	<i>65</i>
<i>Figure A.1: MANTIS project Gantt chart.</i>	<i>87</i>
<i>Figure B.1: Supervisor meeting form, project diary and log (1 of 2).</i>	<i>88</i>
<i>Figure B.2: Supervisor meeting form, project diary and log (2 of 2).</i>	<i>89</i>

List of Tables

<i>Table 1.1: Measurable project objectives and success criteria.</i>	<i>3</i>
<i>Table 2.1: Snort 3.x pros and cons.</i>	<i>10</i>
<i>Table 2.2: Suricata 7.x pros and cons.</i>	<i>11</i>
<i>Table 2.3: Zeek 8.x pros and cons.</i>	<i>12</i>
<i>Table 2.4: Wazuh 4.x pros and cons.</i>	<i>13</i>
<i>Table 2.5: Comparable system strengths and limitations.</i>	<i>16</i>
<i>Table 2.6: Functional requirements with inputs and implementation logic.</i>	<i>20</i>
<i>Table 2.7: Performance and environment targets for final evaluation.</i>	<i>29</i>
<i>Table 3.1: Actor use case associations, include and extend.</i>	<i>39</i>
<i>Table 3.2: Principal MANTIS use case documentation.</i>	<i>40</i>
<i>Table 3.3: Logical to physical persistence mapping.</i>	<i>52</i>
<i>Table 4.1: MANTIS build components and technical constraints.</i>	<i>67</i>
<i>Table 5.1: Test levels mapped to requirements and evidence modules.</i>	<i>68</i>
<i>Table 5.2: Small domain matched algorithm verification results.</i>	<i>70</i>
<i>Table 5.3: Twenty eight representative executed test cases and scenarios.</i>	<i>70</i>
<i>Table 5.4: Terminal execution evidence for test cases TC-01 to TC-28.</i>	<i>76</i>

Abbreviations

AF_PACKET	Linux raw packet socket family
API	Application Programming Interface
ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
CIC	Canadian Institute for Cybersecurity
CLI	Command Line Interface
CSV	Comma Separated Values
DDoS / DoS	Distributed Denial of Service / Denial of Service
FR / NFR	Functional Requirement / Non Functional Requirement
HIDS	Host Based Intrusion Detection System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICMP	Internet Control Message Protocol
IDS / IPS	Intrusion Detection / Prevention System
IoT	Internet of Things
IPv4 / IPv6	Internet Protocol version 4 / version 6
JSON	JavaScript Object Notation
KNN	K Nearest Neighbours
MANTIS	Minimalist Alert Network for Threat Intelligence System
MCC	Matthews Correlation Coefficient
ML	Machine Learning
NIST	National Institute of Standards and Technology

PCAP	Packet Capture file format
PII	Personally Identifiable Information
PPS	Packets Per Second
SIEM	Security Information and Event Management
SQLi	SQL Injection
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TLS	Transport Layer Security
UML	Unified Modelling Language
VLAN	Virtual Local Area Network

1. Introduction

Cybersecurity monitoring is increasingly required on small Linux servers, Internet of Things (IoT) gateways and teaching laboratory machines, yet many established intrusion detection and security monitoring platforms assume access to substantial memory, storage and supporting software. The National Institute of Standards and Technology describes intrusion detection as the monitoring and analysis of system or network events for signs of security incidents (NIST, 2007). This principle remains important at the network edge, where a compromised low cost device can provide an entry point for reconnaissance, denial of service activity or further attacks. The practical challenge is therefore not only to detect suspicious behaviour, but to do so transparently and within the resources available on commodity Linux hardware.

This dissertation presents MANTIS, the Minimalist Alert Network for Threat Intelligence System, as a constructive response to that challenge. MANTIS is a host deployed intrusion detection artefact written with the Python Standard Library and Linux operating system facilities. It combines statistical, behavioural, probabilistic, clustering and reputation based detection in one auditable workflow, then records and communicates alerts through persistent logs and a browser dashboard. This chapter establishes the project background and rationale, states a single aim with directly linked objectives, and summarises the development and evaluation methodology used in the final project.

1.1 Background

Modern networks contain heterogeneous endpoints that generate traffic with changing patterns and different levels of trust. Small Linux servers, Internet of Things gateways and teaching laboratory computers can process sensitive network activity while operating with limited memory, storage and processing capacity. These devices can become targets for reconnaissance, flooding, malicious payloads and communication from known harmful sources. Therefore, network monitoring is required not only in enterprise environments but also on smaller hosts where deploying complex security infrastructure may be impractical.

Signature based tools such as Snort and Suricata remain valuable for recognising documented attack patterns, while platforms such as Wazuh provide broader monitoring and security event management. However, signature based detection is primarily designed around previously defined indicators, while enterprise platforms can introduce deployment and maintenance overhead that is disproportionate for resource constrained hosts. Research also demonstrates that high laboratory accuracy does not necessarily establish operational suitability because live performance is affected by detection latency, memory consumption, throughput and changes in traffic distribution (Sommer and Paxson, 2010; Neto et al., 2023). This creates a need for a lightweight and transparent monitoring approach that combines different detection techniques.

1.2 Rationale

MANTIS is proposed in response to this resource and transparency gap. Its purpose is to investigate whether several understandable detection methods can operate together on commodity Linux hardware without relying on a large external machine learning framework. Combining complementary engines allows the system to examine different indicators rather than depending on a single model or signature source. The system also records the reasoning and evidence associated with alerts so that an administrator can understand why network activity was classified as suspicious.

MANTIS captures IPv4, TCP and UDP traffic through Linux AF_PACKET sockets and routes derived observations to five engines: Z-Score for traffic volume anomalies, K-Nearest Neighbors for scanning behaviors, Multinomial Naive Bayes for suspicious plaintext payload tokens, Mini-Batch K-Means for behavioral novelty and a Bloom Filter for known malicious source reputation. Learned state can be exported to and restored from a JSON brain file, while alerts can be logged, forwarded and inspected through the dashboard. Its scope is intentionally limited to commodity Linux and observable network traffic. MANTIS does not claim to decrypt HTTPS traffic or replace a distributed enterprise SIEM.

1.3 Aim and Objectives

To design, implement and critically evaluate MANTIS as a transparent, low footprint host based intrusion detection system for commodity Linux that integrates five explainable detection techniques and produces actionable security alerts using no external Python runtime packages.

The aim is assessed through six measurable objectives. Success is defined by artefacts and tests, not by listing implemented features.

Table 1.1: Measurable project objectives and success criteria.

ID	Measurable objective	Success criterion
O1	Produce a testable requirements specification linking each detection, persistence and response obligation to an acceptance check.	Table 2.6 and the NFR table exist; each Table 5.3 case names a level, expected outcome and evidence module.
O2	Implement five inspectable engines behind analyse/train/export_brain/import_brain using only the Python Standard Library.	Engine unit tests pass; src/engines has no third party imports; brain round trip restores state (TC-12).
O3	Integrate AF_PACKET or PCAP capture, bounded queuing, parsing, dispatch, CSV/JSON evidence and policy gated iptables response.	Integration tests for truncated frames, privacy redaction and default disabled blocking pass without changing the host firewall.
O4	Provide reproducible training that validates labelled columns, including CIC-IoT2023 derived unique_ports, and writes JSON brain state.	A missing column file raises a clear loader error (TC-28); valid train export import restores engine state.
O5	Evaluate with unit, integration, system and regression tests; treat small sample metrics as functional checks unless a held out corpus is used.	116/116 tests pass (20 August 2026, 4.642 s); Table 5.2 is labelled verification, not field accuracy; MCC is reported with F1.

ID	Measurable objective	Success criterion
O6	Compare MANTIS with Snort, Suricata, Zeek and Wazuh on low footprint inspectable host criteria and state operational bounds.	Chapter 2 contains four system reviews; Section 1.5 and the conclusion state Linux only capture, no TLS inspection and laboratory evaluation.

1.4 Overview of Methodology

The subsections below record how the software was implemented: the Kanban process used to organize the work, the Python Standard Library technology stack, and the laboratory tools used to build and verify MANTIS.

1.4.1 Methodology Framework

An Agile Kanban framework was selected because the artefact comprises separable engines and integration points that could not be specified completely before coding began. Waterfall would have frozen thresholds, feature contracts and dataset mappings before live tests revealed defects such as flood/scan confusion or baseline poisoning. Scrum sprint ceremonies were unnecessary for a solo project. Kanban columns were Backlog, To Do, In Progress (WIP limit 2), Testing (WIP limit 1) and Done. Each card named a deliverable, an acceptance test and an evidence location. Failed evaluation cards were reopened rather than hidden, which preserved the distinction between interim and corrected engine behaviour.

1.4.2 Technology Stack and Languages

The detection runtime is Python 3.6+ using only the Python Standard Library (Python Software Foundation, 2026). Capture uses socket and AF_PACKET; headers are decoded with struct; concurrency uses threading and queue; persistence uses csv and json files rather than a database server; the dashboard uses http.server, socket UDP/JSON and HTML/JavaScript in dashboard_ui. Active defence calls iptables through subprocess. Training and evaluation modules (training.train, evaluation.evaluate) read labelled CSV/JSON, including CIC-IoT2023 derived tables after explicit unique_ports aggregation. There is no pip dependency for the sensor path. Linux is required for live sniffing; Windows is used only for offline training or documentation.

1.4.3 Development Tools and Environments

Source was edited in Visual Studio Code / Cursor with Git for version history. The execution environment is an isolated VirtualBox laboratory: Ubuntu Linux as the sensor host and Kali Linux as the attacker for Nmap, hping3 and SQLmap scenarios. Installation is performed with `src/install.sh` across `apt`, `dnf`, `pacman`, `zypper` and `apk`. Verification uses `python -m unittest discover -s tests`. Packet evidence is inspected with the built in PCAP writer rather than Scapy. The SIEM is opened at `http://localhost:8080`. These tools were chosen because they keep the claim of a zero dependency detector intact while still allowing repeatable tests.

1.5 Limitations

The project is bounded as follows. Live capture and iptables response are Linux specific and require `cap_net_raw/cap_net_admin` or equivalent privilege. Payload analysis inspects plaintext only; TLS content is out of scope. The prototype does not implement dashboard authentication or encrypted UDP. Evaluation uses laboratory traffic, PCAP replay and small domain matched labelled sets; results demonstrate implementation behaviour and must not be read as unrestricted production accuracy. Time and hardware limited comparison to qualitative product analysis of Snort, Suricata, Zeek and Wazuh rather than a shared line rate bake off. IPv6, DPDK/eBPF offload and multi node consensus are excluded.

2. Literature Review

This chapter reviews four peer reviewed studies and four comparable systems, then converts their lessons into the methodology, evaluation strategy and requirement specification for the final artefact. Each paper is discussed in three paragraphs: the first identifies the study, where and when it was published and how it was conducted; the second explains its relevance to MANTIS; the third summarises its findings and the gap it leaves for this project.

2.1 Review of Related Research

2.1.1 Applying Generative Machine Learning to Intrusion Detection: A Systematic Mapping Study and Review

Halvorsen, Izurieta, Cai and Gebremedhin (2024) published this systematic mapping study and review in ACM Computing Surveys, volume 56, issue 10, article 257. Working from Washington State University and Montana State University, the authors define research questions, filter the candidate literature and classify the surviving studies at the intersection of generative machine learning and intrusion detection. The mapped work falls into three uses: adversarial or penetration testing, improvement of scarce and unbalanced training data, and generative models operating directly as detectors.

The study is relevant to MANTIS because it confirms the problems this project starts from: malicious samples are scarce, class distributions are unbalanced, false positives are costly and some attack behaviours cannot be collected safely. MANTIS answers within a different design space. Instead of generating traffic, it keeps five Standard Library engines whose inputs and decisions remain inspectable, records the responsible engine, label and confidence in a common Verdict, and keeps training data, replay traffic, live capture and saved brain state as separate evidence sources so prepared data is never presented as live network proof.

Findings Gap Analysis: The review finds that generative models can strengthen intrusion detection testing and training, but the mapped evidence is heterogeneous: studies generate different artefacts, evaluate different datasets and report incompatible fidelity and classification measures,

and no accepted metric yet exists for the fidelity of generated security data. The remaining gap is the deployment path. The paper measures no packet latency, memory use or safe host response, so a space is left between improved experimental data and a small auditable detector processing real frames on commodity Linux, and MANTIS is built to occupy that space.

2.1.2 CICIoT2023: A Real Time Dataset and Benchmark for Large Scale Attacks in IoT Environment

Neto, Dadkhah, Ferreira, Zohourian, Lu and Ghorbani (2023) published CICIoT2023 in Sensors, volume 23, issue 13, article 5941, from the Canadian Institute for Cybersecurity at the University of New Brunswick. The authors build a documented testbed of 105 Internet of Things devices, execute 33 attacks grouped into seven families together with benign activity, and release both the attack procedures and the extracted traffic features. Five learning methods are then benchmarked at three label granularities after the records are combined, shuffled, split 80 to 20 and normalised.

The dataset is the principal external training and evaluation source for MANTIS, and the paper defines how it must be used responsibly. The loader validates the schema before any record reaches an engine, each detector receives only features whose semantics match its input, and a destination port value is rejected rather than silently substituted for the aggregated unique port count required by the scan detector. The raw capture path is tested separately from the prepared flow path, so benchmark records are never mistaken for live traffic evidence.

Findings Gap Analysis: The benchmark finds strong binary classification across the reported models, while grouped and individual attack classification degrade progressively; the decline in F1 is more informative than accuracy alone. The gap concerns generalisation and deployment. A shuffled row level split can place closely related flows in both training and testing, the controlled topology and uneven class distribution limit external validity, and the extracted CSV features are not the raw Ethernet frames MANTIS parses. The study also measures nothing about queue behaviour, state persistence or alert delivery, which the MANTIS test suite covers instead.

2.1.3 Machine Learning Based Intrusion Detection System: An Experimental Comparison

Hidayat, Ali and Arshad (2023) published this experimental comparison in the Journal of Computational and Cognitive Engineering, volume 2, issue 2, pages 88 to 97. Working across Edinburgh Napier University, the University of Glasgow and Glasgow Caledonian University, the authors preprocess the TON_IoT records, select features using Pearson correlation and a Random Forest model, and compare Decision Tree, KNN and AdaBoost alongside MLP and LSTM under a 75 to 25 split, reporting accuracy, precision, recall, F1 and ROC behaviour.

The paper informed two MANTIS decisions. First, KNN is adopted only as a scan oriented component whose feature vector, scaling, neighbour votes and confidence are all recorded, rather than as a universal intrusion classifier. Second, evaluation must report several measures and computational cost together, because the authors show that the best scoring model is not automatically the most deployable; AdaBoost reaches the highest accuracy but takes substantially longer to train and test. MANTIS therefore reports per engine metrics with the environment and sample sizes stated.

Findings Gap Analysis: The reported accuracies of 99.6, 99.0 and 99.8 percent for Decision Tree, KNN and AdaBoost show that prepared TON_IoT tables separate well. The gap is the distance from a live sensor. A single random split from one source cannot test unseen attack families, capture periods or environments, and the study integrates no packet acquisition, online feature windows, bounded queuing, persistence or response path. Its selected flow columns are also not guaranteed to exist during live AF_PACKET analysis, which is why MANTIS validates every training column explicitly before use.

2.1.4 A False Sense of Security? Revisiting the State of Machine Learning Based Industrial Intrusion Detection

Kus, Wagner, Pennekamp, Wolsing, Fink, Dahlmanns, Wehrle and Henze (2022) presented this study at the 8th ACM Cyber Physical System Security Workshop, published by ACM in New York. Instead of allowing every attack type to appear in both training and testing, their method withholds individual attacks or complete categories from training and then measures whether

several literature derived industrial intrusion detection classifiers still recognise the withheld behaviour on process level datasets.

This paper shaped how MANTIS reports its own results. Functional test passes are kept separate from precision, recall, F1 and MCC; K-Means distance is described as behavioural novelty rather than verified zero day detection; engines can withhold observations already judged anomalous so a sustained attack cannot redefine the baseline as normal; and the conclusion never converts 116 passing software tests into a field accuracy claim. The paper also defines the evaluation MANTIS still requires in future work: withholding attack families, devices and time periods before claiming generalisation.

Findings Gap Analysis: The central finding is that scores near 99 percent under familiar random splits can collapse to detection rates between 3.2 and 14.7 percent for specific withheld attacks, because the evaluated models learn signatures of attacks present in training rather than stable properties of normal behaviour. The gap is constructive rather than solved: the study critiques evaluation practice on industrial datasets but builds no lightweight Linux monitoring artefact and does not close the generalisation problem it exposes, so its warning applies to any small verification set, including the one used in Chapter 5 of this report.

2.2 Comparable Systems

Four established systems were selected because they represent different monitoring approaches: signature inspection, multithreaded network security monitoring, behavioural telemetry and host centred security analytics. Each system is considered separately using the interim report pattern of a short evaluation, an official product or documentation figure, and a dedicated pros and cons table. The comparison focuses on relevance to MANTIS rather than declaring an overall market winner.

2.2.1 Snort 3.x

Snort 3.x is a mature network intrusion detection and prevention system whose rules describe traffic conditions and the action to take when they match. Its rule language, protocol inspection and large community ruleset make it a strong reference for deterministic and explainable detection

(Cisco Talos, 2024). For MANTIS, Snort demonstrates the operational value of auditable rules and inline response. Its limitation is that learning and anomaly detection are not native equivalents of the five trainable or adaptive engines investigated by this project.

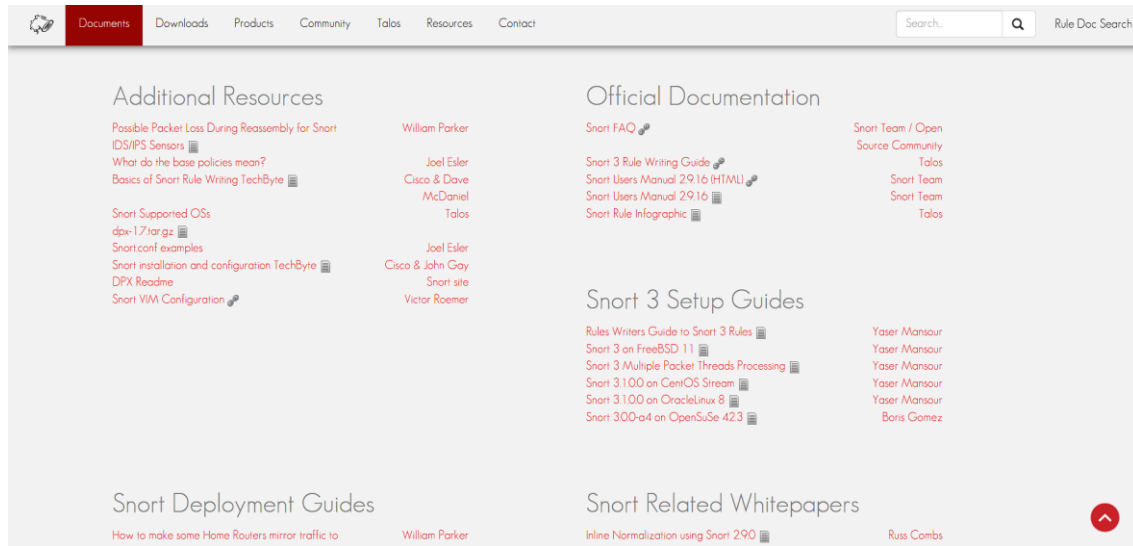


Figure 2.1: Snort product and documentation page (Cisco Talos, 2024).

Table 2.1: Snort 3.x pros and cons.

Pros	Cons
• Mature signature language and rules ecosystem	• No native trainable anomaly engine comparable to MANTIS
• Deep packet and protocol inspection	• Rule quality and maintenance directly affect coverage
• Supports passive detection and inline prevention	• Unknown behaviour may not match an existing signature
• Deterministic matches are straightforward to audit	• Build and deployment are broader than a Standard Library prototype

2.2.2 Suricata 7.x

Suricata 7.x is an open-source network threat detection engine designed for modern multithreaded processing. It supports IDS, IPS and network security monitoring functions and exposes rich EVE JSON events that integrate well with external analytics (OISF, 2024). This structured output is an important design reference for MANTIS telemetry. However, Suricata remains primarily a rules

and protocol inspection platform; machine learning workflows normally sit outside the core engine, whereas MANTIS makes its five small detection models part of the artefact being studied.

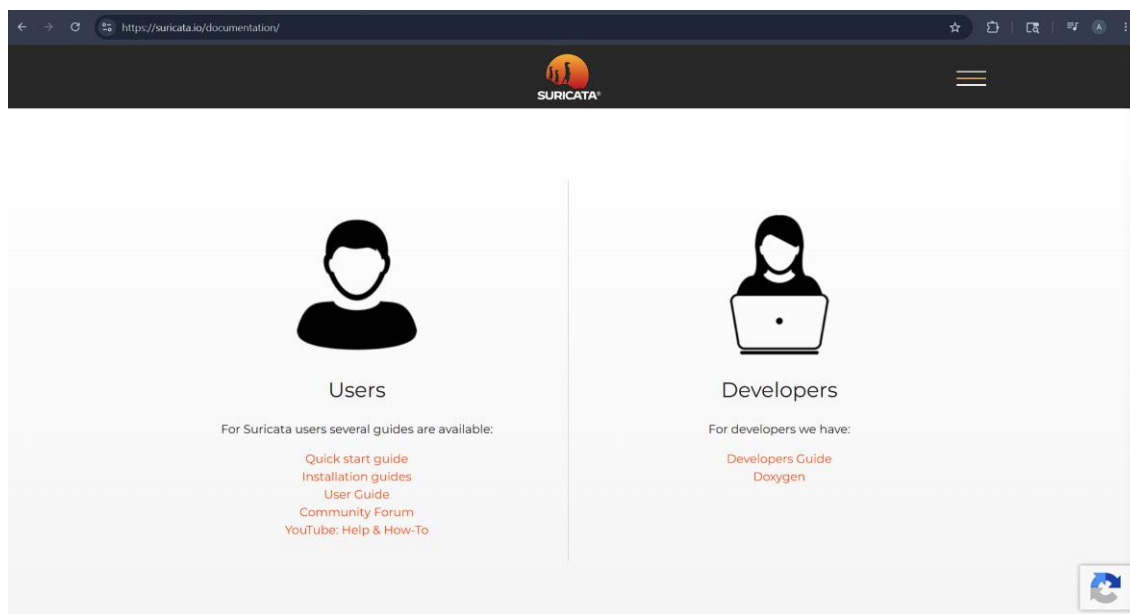


Figure 2.2: Suricata documentation page (OISF, 2024).

Table 2.2: Suricata 7.x pros and cons.

Pros	Cons
• Multithreaded architecture supports high throughput inspection	• Learning normally requires an external analytics pipeline
• EVE JSON provides structured alerts and flow records	• Signature coverage still depends on rules and configuration
• Broad protocol and file inspection capabilities	• Its feature scope is larger than the project's minimalist target
• Can operate as IDS, IPS or monitoring sensor	• Operational tuning is more complex than a small teaching prototype

2.2.3 Zeek 8.x

Zeek 8.x is a network security monitoring framework that converts traffic into detailed protocol logs and events which analysts can extend through its scripting language. Its behavioural visibility and event driven design make it a valuable comparison for anomaly oriented monitoring (Zeek Project, 2026). Zeek's strength is not a single classifier but the context it exposes for analysis. It

does not provide native inline blocking as its central purpose, so response is normally coordinated through another control mechanism or security platform.

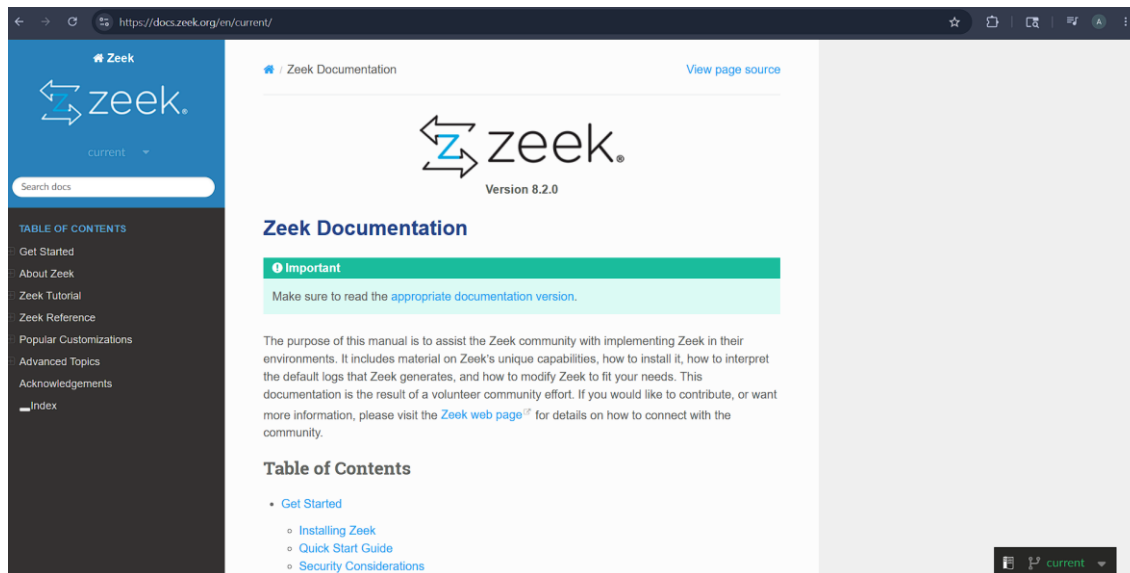


Figure 2.3: Zeek documentation page (Zeek Project, 2026)

Table 2.3: Zeek 8.x pros and cons.

Pros	Cons
• Detailed protocol logs support behavioural investigation	• Inline prevention is not its principal operating model
• Event driven scripting enables custom detection policies	• Effective deployments require scripting and policy expertise
• Strong visibility for incident analysis and hunting	• External components are commonly used for enforcement
• Transparent scripts can be reviewed and versioned	• It does not provide MANTIS style trainable engines as one compact bundle

2.2.4 Wazuh 4.x

Wazuh 4.x is a broader open source security platform built around agents, a central server, an indexer based on OpenSearch and a dashboard. It combines endpoint telemetry, rule based analysis, central management and configurable active response (Wazuh, 2026). These capabilities make it suitable for distributed monitoring, but its purpose and deployment scale differ from a single lightweight network sensor. The comparison is therefore about architectural scope: Wazuh

offers platform integration, while MANTIS concentrates on inspectable detection algorithms and a small experimental runtime.

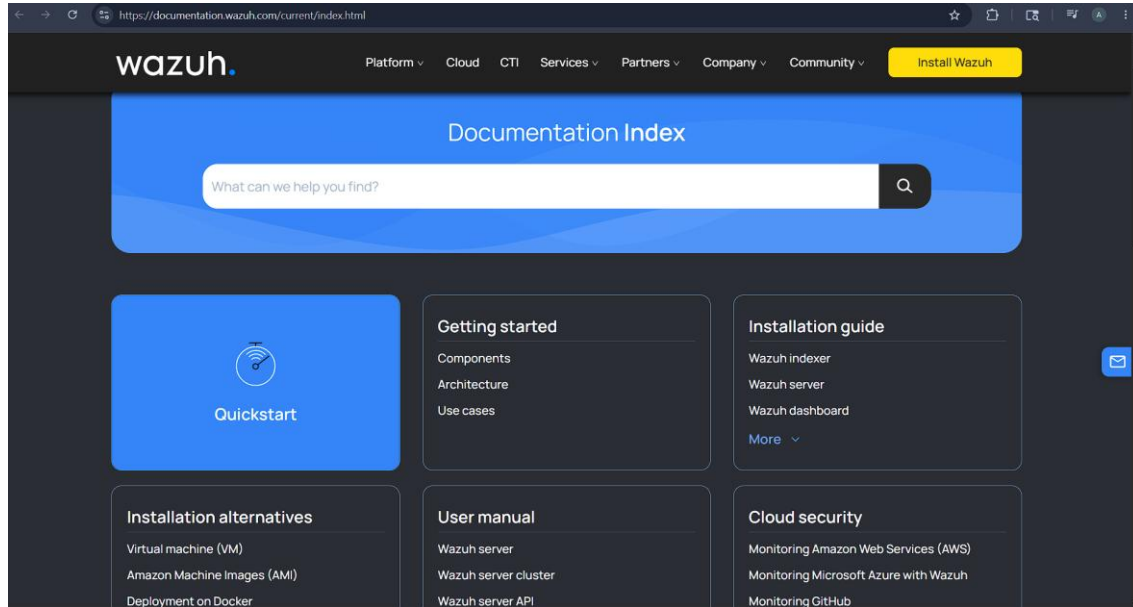


Figure 2.4: Wazuh documentation page (Wazuh, 2026)

Table 2.4: Wazuh 4.x pros and cons.

Pros	Cons
• Centralised management for distributed agents	• Multi component stack has a larger operational footprint
• Rules, decoders, dashboards and active response	• Deployment and maintenance exceed a single sensor prototype
• Searchable security event storage and correlation	• Network anomaly learning is not its sole design focus
• Suitable for wider endpoint and compliance monitoring	• Central services create more configuration and security responsibilities

2.3 Research Methodology

The project uses constructive research methodology, in which knowledge is produced by building and evaluating an artefact that answers a defined practical problem (Crnkovic, 2010). Requirements are derived from literature, comparable systems and the deployment context; the

artefact is then built and evaluated against those requirements, creating a traceable chain from research challenge to design decisions, implementation evidence and final critique.

The methodology combines artefact construction with empirical evaluation. Quantitative evidence includes unit and integration results, per engine confusion matrices, precision, recall, F1 score, latency, throughput and resource measurements. Qualitative evidence is limited to structured usability observations and architectural critique. This mixed evidence is used to judge fitness for the stated low resource research context, not to claim equivalence with mature commercial or community platforms.

2.4 Implementation Traceability

A pragmatic philosophy is appropriate because the research question is concerned with whether a transparent, low footprint detector can be built and evaluated usefully under practical constraints. The project therefore selects methods according to the evidence needed for that question. Mathematical inspection explains how decisions are produced, tests show whether components behave as specified, and measurements show how the integrated artefact performs in a stated environment.

Pragmatism does not remove methodological control. Dataset origin, preprocessing, configuration, operating system, hardware and test conditions are reported so that results can be interpreted and repeated, and claims are restricted to the tested implementation and environment. Conditions that were not evaluated, such as unknown operational traffic, encrypted payloads and adversarial evasion, are treated as limitations rather than inferred strengths.

2.5 Development Methodology

Development used Kanban with a work in progress limit of two coding cards and one testing card. Each card named a module, an acceptance test and an evidence file. Engine cards mapped to `src/engines` and `test_engines.py`; capture and parser cards mapped to `sensor.py`, `parser.py` and `test_integration.py`; SIEM cards mapped to `siem.py` and `test_siem_api.py`. Failed evaluation cards returned to In Progress.

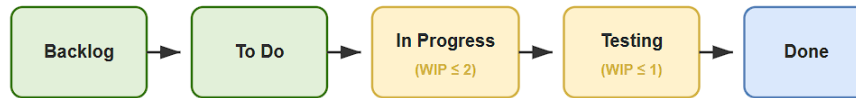


Figure 2.5: Kanban development workflow from the project plan.

The board was refined during implementation: engine defect cards were reopened after evaluation, SIEM tests were added and privacy checks moved to the dispatcher. The full project Gantt chart is reproduced in Appendix A, and progress against it is evidenced by the mapped modules and the 116 test suite.

Each item is completed through a short build and verify cycle: define an acceptance condition, implement the smallest coherent change, test it at the appropriate level, record evidence, and update the dissertation or project log. The approach keeps the final artefact linked to the proposal and interim milestones while allowing requirements to be corrected when the code, platform or evaluation data contradicts an earlier assumption.

2.6 Evaluation Strategy

Evaluation is organised in layers. Unit tests examine packet parsing, engine calculations, configuration and state serialisation. Integration tests examine the path from parsed traffic through engine verdicts, dispatch, logging, UDP telemetry, dashboard ingestion, optional response and PCAP export. System tests use controlled replay and live loopback traffic to verify end to end behaviour without exposing public networks or relying on uncontrolled attack activity.

Detection quality is reported separately for applicable engines using true positives, false positives, true negatives, false negatives, precision, recall and F1 score. Runtime evaluation records latency, traffic rate, CPU, memory and test environment together, because an isolated latency value is not interpretable without load and hardware. Training and testing data are separated before model fitting. Results that use small synthetic samples are labelled as functional checks, while claims about detection effectiveness require an adequately sized held out dataset.

2.7 Methodology for Comparable Analysis

The comparable analysis uses five project defined criteria: anomaly detection support, active response, suitability for a low footprint deployment, transparency of decision logic, and support for training or adaptation. Each system is assigned a qualitative alignment score from 1 to 5, where 1 indicates weak alignment with the MANTIS research context and 5 indicates strong alignment. The chart is a requirements oriented rubric based on documented capabilities; it is not a throughput benchmark or an independent product ranking.

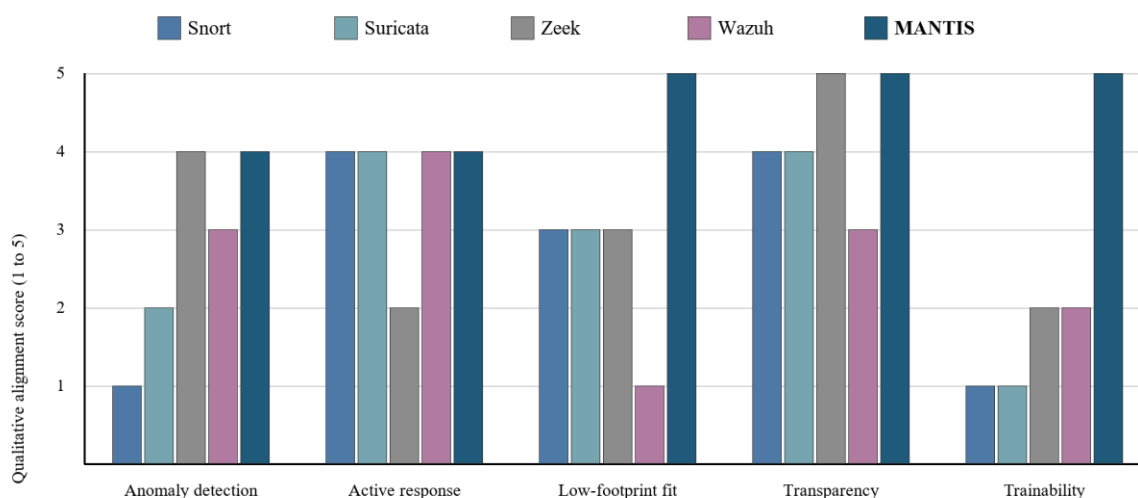


Figure 2.6: Qualitative comparable system functionality ranking using the project rubric.

Table 2.5: Comparable system strengths and limitations.

System	Principal strength	Limitation in the MANTIS context
Snort 3.x	Mature rule language and deep packet inspection	No native learning or anomaly model; deployment centres on rule maintenance
Suricata 7.x	Multithreaded inspection with structured EVE JSON output	Learning normally depends on an external analytics component
Zeek 8.x	Rich behavioural telemetry and scriptable protocol analysis	Detection and response policies require scripting and external enforcement

System	Principal strength	Limitation in the MANTIS context
Wazuh 4.x	Centralised agent management, correlation and active response	Multi component platform is broader and heavier than a small standalone sensor
MANTIS	Five inspectable engines, training, state persistence and optional response in one prototype	Narrower protocol coverage and less operational maturity than established platforms

The comparison shows why MANTIS is a focused experimental artefact rather than a replacement for the four systems: Snort and Suricata offer greater signature and protocol maturity, Zeek provides richer behavioural telemetry, and Wazuh offers broader central management. The final evaluation therefore tests only the low footprint, inspectable criteria deliberately built into the MANTIS scope and acknowledges the prototype's narrower coverage and lower operational maturity.

2.8 Requirement Specification

The literature gaps and comparable system findings are converted below into requirements that can be traced to implementation and evaluation evidence. Functional requirements state what the artefact must do; non functional requirements state the qualities and constraints under which those functions should operate. Measurements are presented as targets at this stage and will be reported as results only after final testing.

2.8.1 Problem Domain

MANTIS addresses network monitoring on commodity Linux hosts where a learner or small operator needs inspectable detection without deploying a full security platform. The problem is defined by the following linked constraints:

- The runtime should remain small enough for a modest virtual machine, teaching laboratory, or edge Linux host, without requiring third-party Python packages for detection.
- Signature-only monitoring may miss behaviour without a prepared rule, while opaque models make it difficult to explain why an alert was generated.

- Capture, alerting, persistence, evaluation, and optional response should form one reproducible workflow, while privileged actions and sensitive payload collection remain controlled.
- Reconnaissance, volumetric anomalies, malicious payloads, behavioural novelty, and known malicious indicators require complementary detection mechanisms rather than a single universal classifier.
- Operators should be able to trace every alert to observable features, a stated threshold or learned state, and a documented decision path.
- Labelled training data, replay traffic, and live network behaviour differ, so the evidence should distinguish implementation verification from claims of real-world accuracy.
- Packet payloads and addresses may expose personal or confidential information, so data minimisation, redaction, bounded retention, and controlled export are required.
- Learned engine state and audit evidence should survive an orderly restart without silently accepting corrupted or incompatible brain data.
- Claims about detection, response, and resource use should be reproducible through traceable tests, explicitly defined environments, and reviewable output artefacts.

2.8.2 Proposed Solution Strategy

Each constraint in Section 2.8.1 is answered below by a corresponding solution decision. Keeping the same order and number makes the link from the problem domain to implementation explicit:

- MANTIS uses Python Standard Library detection code, bounded queues and buffers, compact online state, and configurable retention limits so that it can operate on a modest Linux host.
- The system combines transparent statistical, supervised, probabilistic, clustering, and membership methods instead of relying solely on prepared signatures or an opaque end-to-end model.
- Capture, parsing, five-engine analysis, alert dispatch, persistence, SIEM delivery, reporting, and optional active response are connected through a single producer-consumer workflow with controlled privileges.
- Complementary mechanisms are assigned to the principal threat patterns, with Z-Score detecting volumetric anomalies, KNN identifying reconnaissance, Naive Bayes analysing

malicious payloads, K-Means recognising behavioural novelty, and Bloom Filter checking known malicious indicators.

- Each engine returns a common Verdict containing the responsible engine, label, confidence, threat flag, and decision details, while thresholds, features, and learned states remain inspectable.
- Training data, controlled PCAP replay, live observations, and evaluation artefacts are separated, while dataset schemas are validated and each result is clearly identified as evidence of either implementation behaviour or wider detection accuracy.
- Retained payloads are minimised, credentials and email addresses are redacted before persistence or telemetry, packet storage is bounded, and evidence can only be exported through controlled functions.
- All five engine states are exported to JSON during an orderly shutdown, while only compatible brain data is loaded at startup and the system safely returns to an initialised state when a valid snapshot is unavailable.
- Requirements are traced to unit, integration, system, regression, and training tests, with reviewable outputs preserved and accuracy, latency, CPU usage, and memory consumption measured only under an explicitly documented environment and workload.

Together, these nine decisions provide one implementation response for every problem domain constraint. The mapping also separates completed technical controls from performance and field accuracy claims that still require controlled measurement.

2.8.3 Alternative Solution Strategy

Three alternative approaches were considered but not selected as the main dissertation artefact:

- Extending Snort or Suricata with an external learning component would provide mature packet inspection, but the research contribution would shift toward integration with a large existing engine rather than implementation and evaluation of transparent algorithms.
- Using scikit-learn, ONNX Runtime or a neural framework would shorten model development and support more complex algorithms, but it would introduce runtime dependencies and reduce the project's emphasis on first principles, inspectable implementation.

- Deploying Wazuh would provide broader endpoint management and response, but its multi component platform addresses a different operational scale from the single host experimental sensor defined by the aim.

These alternatives remain valid engineering choices outside the dissertation boundary. The selected strategy is justified only for the stated research purpose: to examine what can be achieved with a compact Standard Library runtime and five visible decision mechanisms.

2.8.4 Functional Requirements

Table 2.6 records the implemented functional requirements. Inputs and logic are taken from the MANTIS source modules rather than from an aspirational specification.

Table 2.6: Functional requirements with inputs and implementation logic.

Sub Category	Goal / Objective	Required Inputs	Implementation Details / Logic
Packet acquisition	Capture live Ethernet frames or replay a finite PCAP without stopping the analyser on unsupported frames.	Linux interface name or PCAP path; AF_PACKET raw socket or libpcap format file; optional VLAN/ICMP frames.	SensorNode binds SOCK_RAW/AF_PACKET or opens the PCAP stream. PacketParser unpacks Ethernet/IPv4/TCP/UDP/ICMP with struct; truncated frames are rejected and counted, not fatal.
Pipeline decoupling	Prevent capture stalls when classification is slower than arrival rate.	Raw frame bytes; configured queue size; producer and analyser threads.	A bounded queue. Queue receives frames from the sniffer. On overflow the sniffer drops and records loss. The analyser dequeues, parses and fans observations to the engines.

Sub Category	Goal / Objective	Required Inputs	Implementation Details / Logic
Five engine classification	Detect volumetric floods, reconnaissance, malicious plaintext payloads, behavioural novelty and known bad sources through one contract.	Per interval PPS and unique port counts; payload bytes; source IP; trained or seeded engine state.	Each engine implements analyse/train/export_brain/import_brain. Interval engines (Z-Score, KNN, K-Means) run once per second; Bloom and Naive Bayes run per packet. A Verdict {label, confidence, detail, is_threat} is returned.
Training and brain persistence	Fit applicable engines from labelled data and restore inspectable state after restart.	CSV/JSON/CIC-IoT2023 derived tables with validated columns (unique_ports, packet_count, payload, label); brain JSON path.	training.train and Loader reject illegal mappings such as raw Dst Port as unique_ports. Compatible sections of the newest brain file are imported at start; export occurs on orderly shutdown.
Alert construction	Produce an auditable alert that an operator can explain.	Engine Verdict; packet 5 tuple; optional payload excerpt; MITRE mapping tables.	AlertDispatcher deduplicates, enriches ATT&CK technique/severity/risk, truncates excerpts to 64 bytes and redacts PII before any sink is written.
Persistence and telemetry	Retain a forensic trail and notify a SIEM without blocking capture.	Sanitised alert record; CSV path; UDP host/port default 9999.	Logger appends a locked CSV row. UDPSender emits JSON datagrams. Failures are logged; they do not terminate the analyser.

Sub Category	Goal / Objective	Required Inputs	Implementation Details / Logic
Operator interface	Display current alerts, evidence and exports in a browser.	HTTP GET on port 8080; in memory alert buffer; packet store identifiers.	siem.py serves HTML/JSON from a standard library HTTP server. Malformed UDP is discarded. The dashboard is a local prototype and is not authenticated.
Optional active defence	Block a confirmed source only under explicit operator policy.	--block flag; --block-on categories; whitelist; source IP; iptables availability.	ActiveDefense inserts iptables -I INPUT 1 -s IP -j DROP only when enabled, category permitted, not whitelisted and not already blocked. flush() removes MANTIS rules on exit.
Evidence export	Support post incident review and external analysis.	Session alerts; bounded packet store; shutdown event.	ReportGenerator writes forensic HTML. Packet evidence can be exported as a valid PCAP. Stores are bounded to prevent unbounded RAM growth.
Evaluation reporting	Produce reproducible confusion matrix metrics for applicable engines.	Held out labelled samples; imported brain state; output path.	evaluation.evaluate writes TP/FP/TN/FN, precision, recall, F1 and related files. Small synthetic sets are labelled as functional checks, not field accuracy.
Configuration and shutdown	Control runtime behaviour and release resources cleanly.	CLI/config options for interface, ports, brain, logs, PCAP and defence; stop signal.	Config supplies defaults. Shutdown stops threads, closes sockets, exports brain state and completes the HTML report.

- FR1: Capture Ethernet frames on a configurable Linux interface using AF_PACKET, while supporting offline PCAP replay when live capture is unavailable.

- FR2: Parse the Ethernet, IPv4, TCP and UDP fields required by the detection pipeline and minimise retained payload data according to configuration.
- FR3: Decouple capture and analysis through a bounded queue and record dropped frame information when the analyser cannot keep pace.
- FR4: Apply the Z-Score, KNN, Multinomial Naive Bayes, Mini-Batch K-Means and Bloom Filter engines through a common analysis contract.
- FR5: Train applicable engines from supported labelled CSV or JSON data, validate required columns, and reject incompatible feature mappings with a clear error.
- FR6: Export and import the state of all five engines as JSON safe brain data, including controlled loading at sensor start and export at shutdown.
- FR7: Create a structured alert containing time, source, destination, protocol, engine, verdict, confidence and a privacy limited payload excerpt where applicable.
- FR8: Persist alerts to an append only CSV audit log and forward structured JSON telemetry to a configurable UDP endpoint, defaulting to port 9999.
- FR9: Provide a browser dashboard on a configurable HTTP port, defaulting to 8080, for current alerts, severity, packet store evidence and permitted exports.
- FR10: Support optional iptables blocking of an alerted source only when active defence is explicitly enabled, and avoid duplicate rules for the same address.
- FR11: Generate a forensic HTML report from collected alerts during orderly shutdown.
- FR12: Evaluate applicable engines using confusion matrix counts, precision, recall and F1 score, with results written to reproducible output files.
- FR13: Expose configuration through documented command line and configuration file options, including interface, remote host, ports, brain path, logs and PCAP capture.
- FR14: Export a valid PCAP representation of captured or replayed packet evidence that can be opened by standard analysis tools.
- FR15: Decode supported VLAN tagged and ICMP frames, and reject truncated or unsupported frames without terminating the analyser.
- FR16: Maintain a bounded in memory packet store for dashboard inspection and PCAP export without unbounded memory growth.

- FR17: Enrich alerts with mapped ATT&CK technique, severity, confidence and bounded risk score while preserving the originating engine rationale.
- FR18: Expose alert, packet, statistics, configuration and export data through the local SIEM HTTP interface and reject malformed UDP telemetry safely.
- FR19: Enforce response policy through enabled state, whitelist and duplicate rule checks, and record whether an alert was flagged or blocked.
- FR20: Load threat intelligence indicators from a documented feed format, ignore comments and report missing or invalid feeds without crashing.
- FR21: Attribute interval based alerts only when a source exceeds the dominance threshold, preventing distributed traffic from being assigned to one host.
- FR22: Perform orderly shutdown by stopping worker threads, closing capture and output resources, exporting brain state and completing forensic reporting.

2.8.5 Non-Functional Requirements

- NFR1 Performance: Meet the latency, CPU and memory targets in Table 2.7 under a documented reference load and environment.
- NFR2 Transparency: Keep each engine's mathematics, thresholds, features and state visible in source code and report per engine results.
- NFR3 Portability: Require only the Python Standard Library for detection logic; use documented Linux facilities for live capture and response; keep offline training and replay usable on other operating systems where supported.
- NFR4 Security: Disable active defence by default, validate externally supplied paths and data, minimise payload retention, and avoid exposing dashboard or UDP services directly to an untrusted network.
- NFR5 Reliability: Bound queues and buffers, handle malformed packets and telemetry without terminating the service, and preserve valid logs and brain data across controlled shutdown.
- NFR6 Maintainability: Implement engines as separate modules sharing analyse, train, export_brain, import_brain and reset operations, with automated tests for their contracts.

- NFR7 Usability: Provide documented installation, command help, clear configuration defaults and operator readable alerts without requiring an external database or web framework.
- NFR8 Reproducibility: Record dataset, split, configuration, software version, hardware and test commands for every reported experiment.

2.8.6 Goals of Implementation

The implementation goal is one coherent, inspectable and testable MANTIS prototype rather than a collection of disconnected algorithms. Capture, parsing, five detection mechanisms, alert dispatch, privacy controls, state persistence, evidence export, evaluation and the four page operator console exchange data through explicit interfaces. Completion is judged through the following goals, each linking an input, an implemented responsibility and verification evidence:

- Acquisition goal: accept live Ethernet frames through Linux AF_PACKET and finite offline PCAP input, validate supported Ethernet, VLAN, IPv4, TCP, UDP and ICMP structures, and reject malformed or unsupported frames without stopping the analyser.
- Detection goal: route semantically correct features to five complementary engines, namely Z-Score, KNN, K-Means, Naive Bayes and Bloom Filter, and return a common Verdict containing the engine, label, confidence, threat state and decision detail.
- Operational goal: use bounded queues, packet history and alert buffers so capture cannot grow memory without limit; keep active defence disabled by default and issue an iptables request only when the threat category, confidence, whitelist and configuration policy all permit it.
- Evidence and privacy goal: redact sensitive payload content before persistence or telemetry, write reviewable CSV and JSON evidence, permit controlled PCAP and JSON export, and expose ATT&CK, severity and risk context without using colour as the only meaning.
- Continuity goal: export all engine state during orderly shutdown, load only compatible brain data at startup, recover safely when no valid snapshot exists, and keep training, live observation and evaluation artefacts distinguishable.
- Interface goal: implement all four dashboard pages. Overview must show host health, recent events and engine activity; Packets must support filtering, pause, detail and export; Alerts must present the complete in memory threat log; Architecture must explain the five engine pipeline and response path.

- Verification goal: trace requirements to unit, integration, system, regression and training tests, use controlled loopback and replay traffic, and report software correctness separately from detector effectiveness, latency, CPU and memory measurements.
- Scope goal: maintain a Python Standard Library detection runtime suitable for a modest Linux host and state clearly that authentication, multi user case management, durable database search, encrypted payload inspection and production scale certification remain outside the implemented boundary.

Together, these goals define implementation success as traceable behaviour rather than the presence of source files. A goal is treated as completed only when the relevant module, observable output and verification evidence agree. The interface figures in Section 3.7 follow the same rule by showing every implemented page and excluding functions that are not present in the current system.

2.8.7 System Workflow Diagrams

Figure 2.7 represents the conventional workflow used as the problem reference: multiple security data sources feed an alert queue, after which analyst correlates evidence and applies a response manually. Figure 2.8 represents the proposed MANTIS workflow, linking training and brain state loading to packet capture, five engine analysis, alert dispatch, dashboard presentation and optional iptables response. The diagrams retain the workflow presentation used in the interim report.

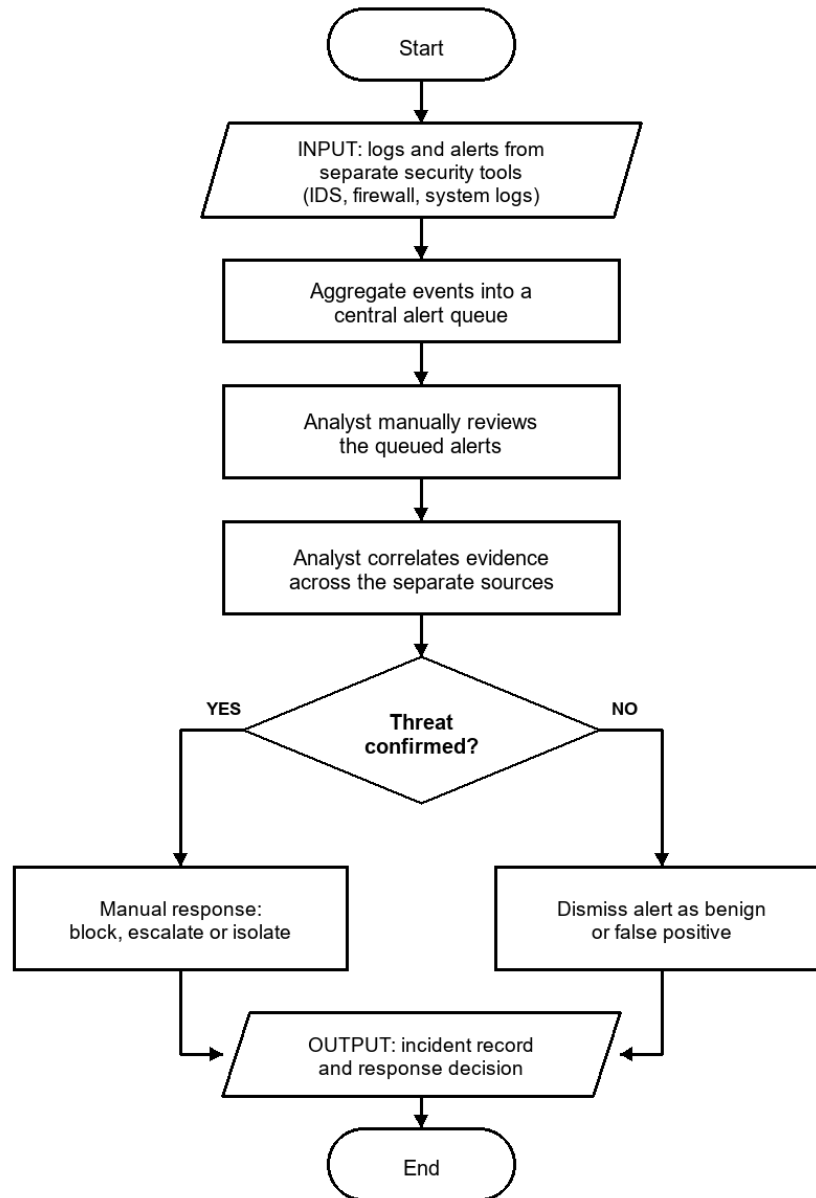


Figure 2.7: Current monitoring and manual response workflow.

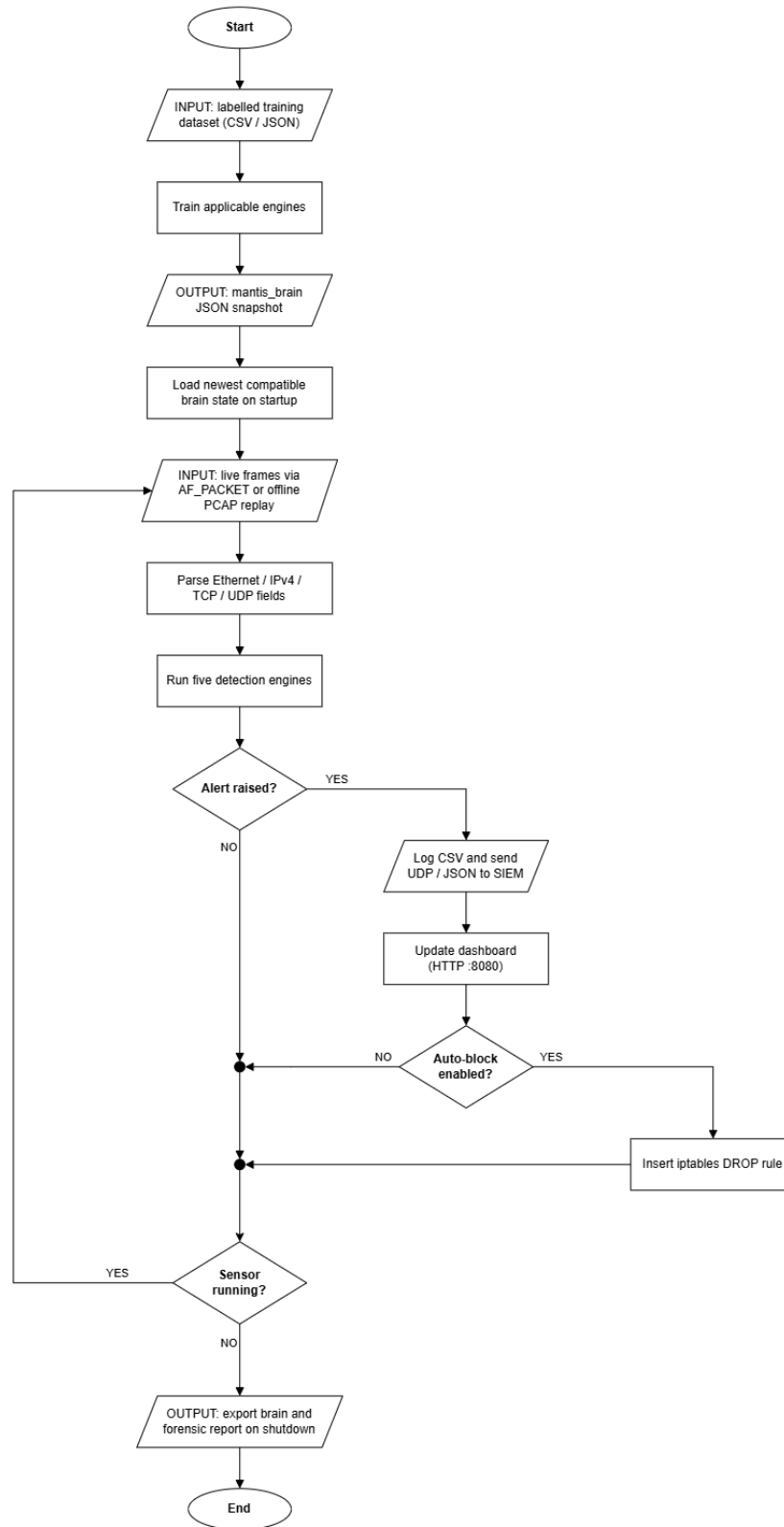


Figure 2.8: Proposed end to end MANTIS workflow.

2.8.8 Performance and Environment Requirements

Live capture requires a Linux host that provides AF_PACKET and sufficient privilege or capability for raw sockets. The reference environment will record processor, memory, operating system version, Python version, interface, traffic source and test duration. The detection runtime uses the Python Standard Library; iptables and Linux capabilities are operating system facilities rather than Python package dependencies. Offline training, evaluation and PCAP replay can be exercised separately where live AF_PACKET capture is unavailable.

Table 2.7: Performance and environment targets for final evaluation.

Requirement	Target	Verification method
Packet processing latency	p95 below 15 ms in the stated test environment	Timed replay and live loopback benchmark
Memory footprint	Sensor process below 100 MB resident memory	Repeated process sampling during controlled traffic
CPU utilisation	Below 25% of one reference vCPU at the stated load	Process monitoring with traffic rate recorded
Reliability	No crash or corrupted alert file during a 30 minute run	Sustained replay, shutdown and restart checks
Portability	Live capture on supported Linux; offline training and replay on other operating systems	Installation and smoke test matrix
Security	Active response disabled by default; captured payload minimised	Configuration review and negative tests

Table 2.7 defines evaluation targets, not completed results. The final discussion will report measured values with sample size, load and environment, and will identify any unmet target. This prevents interim observations or single loopback measurements from being presented as general performance claims.

2.9 Algorithms Implemented

The three algorithms below retain the interim report's documentation pattern. Each is defined, expressed mathematically, described as a procedure, represented in pseudocode and linked to a flowchart. Mini-Batch K-Means and Bloom Filter are documented with the remaining system design material because their state update and persistence behaviour is closely connected to that chapter.

2.9.1 Z-Score Statistical Engine

The Z-Score engine monitors a packet rate observation x relative to an online estimate of the normal mean and standard deviation. It raises a volumetric anomaly indication when the absolute standardised deviation exceeds the configured threshold after sufficient baseline observations.

Mathematical Model

$z = (x - \mu) / \sigma$, alerting when $|z| > 3$. Mean μ and variance σ are updated online via Welford's incremental method.

$$z = (X - \mu) / \sigma$$

Step by Step Procedure

1. Read the current packets per second observation.
2. Update the running count, mean and second moment using Welford's equations.
3. Derive the standard deviation when enough observations exist.
4. Calculate the absolute Z-Score and compare it with the configured threshold.
5. Return a normal or flood oriented verdict with the supporting score.

Pseudocode

INPUT current observation x and threshold τ

UPDATE n , mean μ and second moment $M2$

IF $n < \text{minimum_samples}$ OR $\text{standard_deviation} = 0$: RETURN Normal

$z = (x - \mu) / \text{standard_deviation}$

IF $\text{absolute}(z) > \tau$: RETURN FloodAlert with score

RETURN Normal

Flowchart

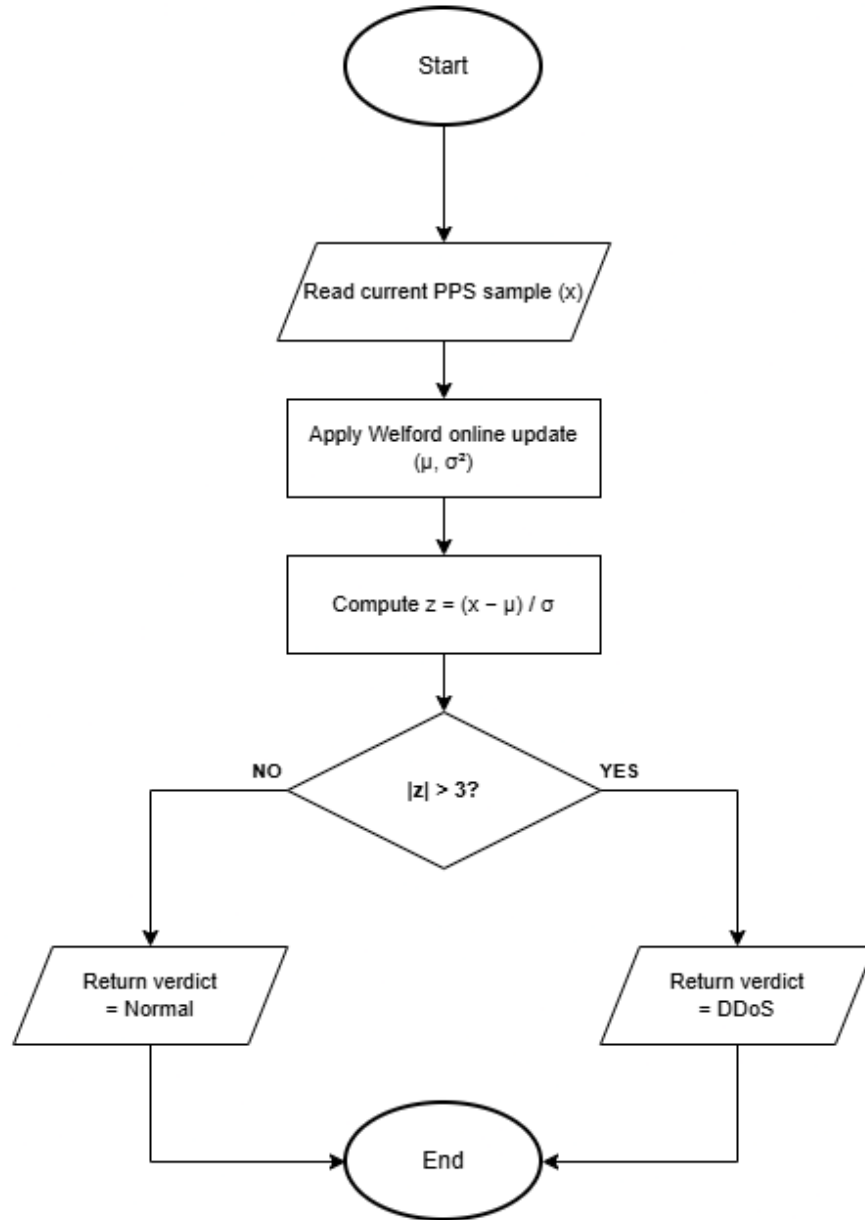


Figure 2.9: Z-Score statistical engine flowchart.

2.9.2 K-Nearest Neighbours (KNN) Engine

The KNN engine is a supervised, instance based classifier used for scan oriented observations. A query vector is compared with labelled training vectors, and the majority label among the k closest neighbours determines the predicted class.

Mathematical Model

Euclidean distance $d(p,q) = \sqrt{\sum_i (p_i - q_i)^2}$. Majority vote of $K=3$ nearest labelled neighbours assigns Normal vs Scan.

Step by Step Procedure

1. Construct the live feature vector, including the configured scan oriented counts.
2. Validate that a compatible labelled training set is available.
3. Calculate Euclidean distance to every stored training vector.
4. Select the k vectors with the smallest distances and count their labels.
5. Return the majority class with vote based confidence and distance evidence.

Pseudocode

```
INPUT query vector q and labelled training data D
distances = [(euclidean(q, vector), label) for each item in D]
neighbours = first k items after sorting distances
prediction = majority_label(neighbours)
confidence = votes_for_prediction / k
RETURN prediction, confidence and neighbour distance
```

Flowchart

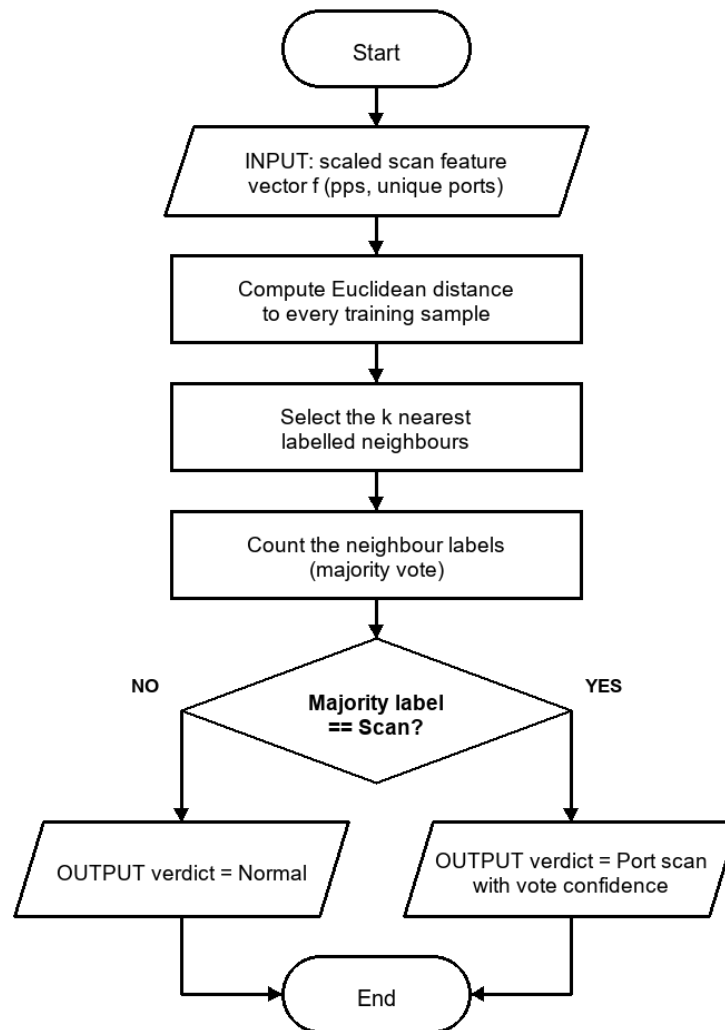


Figure 2.10: K-Nearest Neighbours engine flowchart.

2.9.3 Multinomial Naive Bayes Engine

The Multinomial Naive Bayes engine classifies tokenised, unencrypted payload excerpts using class priors and token likelihoods learned from labelled examples. It is intentionally limited to visible payload text and cannot inspect encrypted application content.

Mathematical Model

$\log P(\text{class}|\text{tokens}) = \log P(\text{class}) + \sum_i \log P(\text{token}_i|\text{class})$; Laplace smoothing prevents zero counts on unseen tokens.

Step by Step Procedure

1. Decode the privacy limited payload excerpt and tokenise visible text.
2. Initialise each class score with its log prior probability.
3. Add the smoothed log likelihood of each token for each class.
4. Select the class with the largest score and derive confidence from the class scores.
5. Return a payload alert verdict only when the malicious class and threshold conditions are satisfied.

Pseudocode

INPUT payload bytes and trained class statistics

tokens = tokenise(decode(payload excerpt))

FOR each class c: score[c] = log_prior[c]

 FOR each token w: score[c] += log(smoothed_probability(w, c))

prediction = class with maximum score

IF prediction is Malicious AND confidence $\geq$ threshold: RETURN PayloadAlert

RETURN Safe

Flowchart

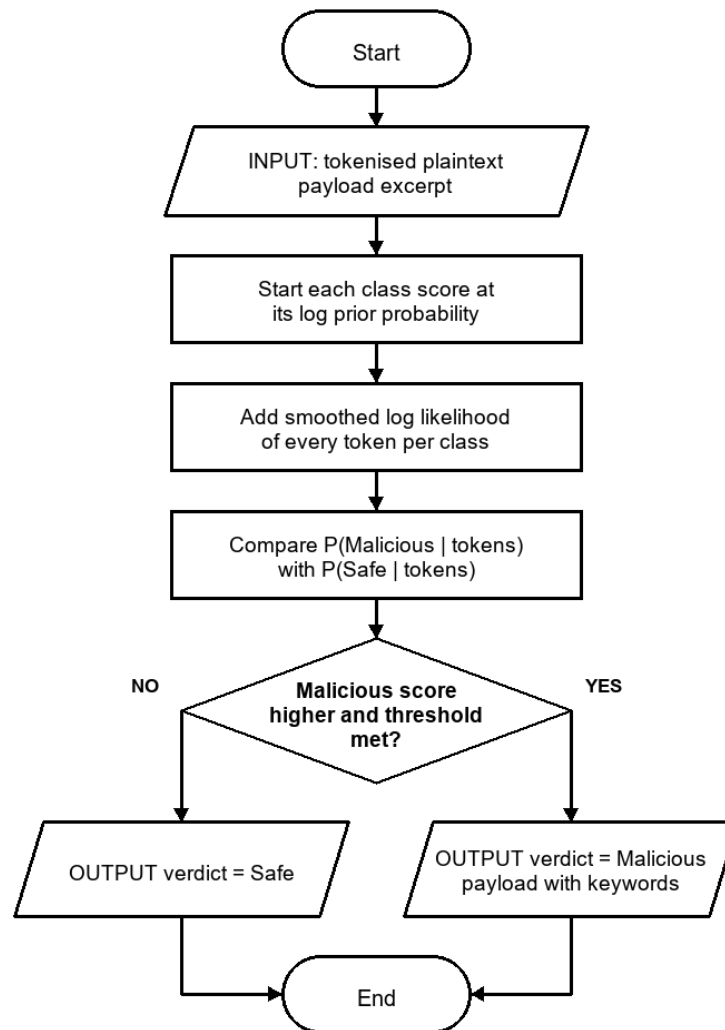


Figure 2.11: Multinomial Naive Bayes engine flowchart.

3. System Analysis and Design

System analysis converts the requirements established in Chapter 2 into actors, interactions, data structures and decision paths, grounded in the implemented MANTIS modules. Each diagram answers a different question: the use case identifies user goals, the sequence diagram shows message order, the class diagram allocates responsibility, the activity diagram shows control flow, and the data schema explains how evidence is retained.

The design keeps acquisition, parsing, detection, dispatch, persistence and presentation separate. A bounded producer consumer queue prevents packet capture from being tied directly to analysis time, while a common Verdict contract allows five different algorithms to feed the same dispatcher. Detection code cannot change the firewall directly; an explicit policy check controls the optional response path.

3.1 Use Case

A use case represents a goal achieved through interaction between an actor and the system. In MANTIS the primary actor is the Security Administrator, who starts monitoring, investigates alerts, manages learned state and requests evidence exports. The Host Operating System supplies frames and enforces an approved firewall rule, while an External SIEM may receive structured telemetry. These actors are outside the MANTIS system boundary even when they run on the same host. Figure 3.1a shows this abstract goal level with every actor placed on the left of the system boundary.

Figure 3.1b refines the abstract model to level 1 detail. Include relationships mark behaviour that always occurs, capturing and analysing traffic during live monitoring, while the extend relationship attaches blocking only when a verdict is raised and the block policy is enabled. This distinction prevents the diagram from implying that every alert automatically blocks a source.

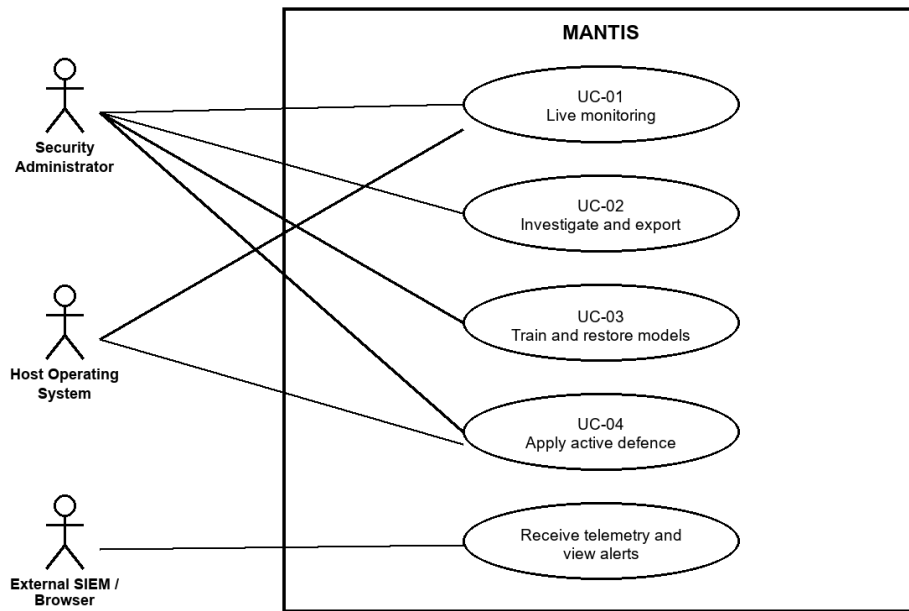


Figure 3.1a: MANTIS abstract goal level use case model.

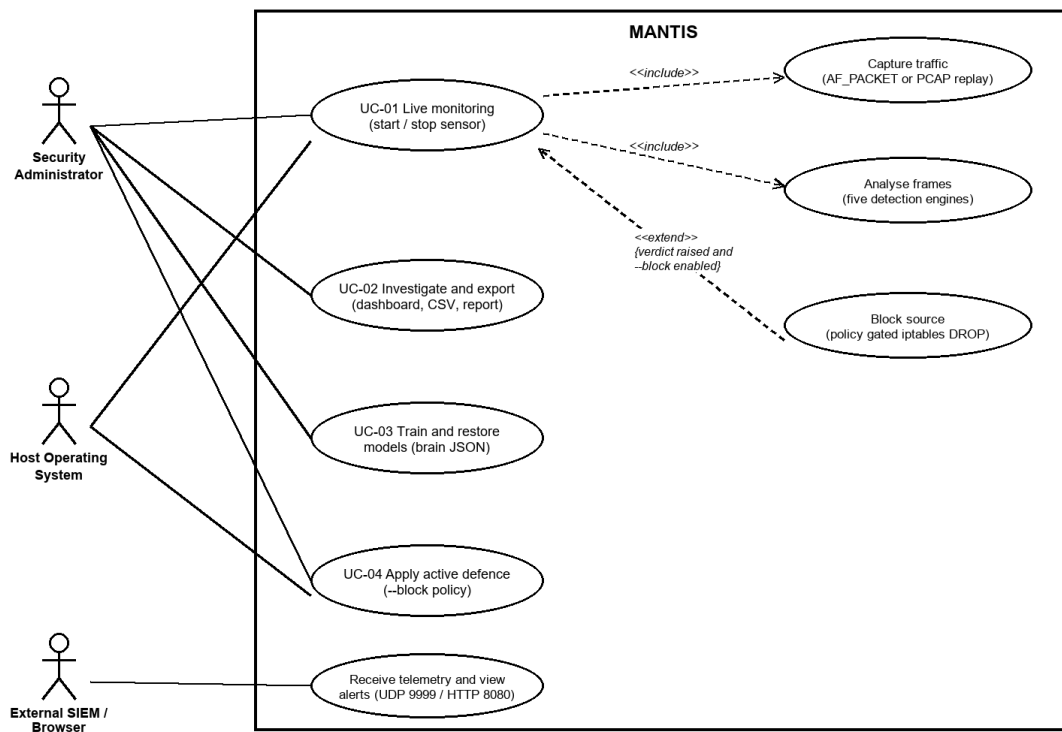


Figure 3.1b: MANTIS level 1 use case model with include and extend detail.

3.1.1 Use Case Documentation

Table 3.1 states the actor associations the use case diagram shows: a solid association between each actor and the use cases it initiates or supports, include for behaviour that always occurs during monitoring, and extend for optional blocking. Table 3.2 then documents the preconditions, main path and exceptions for each principal use case.

Table 3.1: Actor use case associations, include and extend.

Actor or included behaviour	Use case	Association	Meaning in MANTIS
Security Administrator	UC-01 Live monitoring	Association (primary)	Starts and stops the sensor; include Capture Traffic and Analyse Frames.
Security Administrator	UC-02 Investigate and export	Association (primary)	Uses the local dashboard and evidence export.
Security Administrator	UC-03 Train and restore models	Association (primary)	Runs train.py and supplies labelled CSV/JSON.
Security Administrator	UC-04 Apply active defence	Association (primary)	Enables --block; does not occur by default.
Host Operating System	UC-01 Live monitoring	Association	Supplies Ethernet frames via AF_PACKET and process privileges.
Host Operating System	UC-04 Apply active defence	Association	Enforces iptables DROP when the dispatcher requests it.
External SIEM / browser	Receive telemetry / view alerts	Association	UDP JSON to port 9999; HTTP GET on port 8080.

Actor or included behaviour	Use case	Association	Meaning in MANTIS
Capture Traffic	UC-01	Include	Always executed during live or replay monitoring.
Analyse Frames	UC-01	Include	Always executed after a supported frame is parsed.
Block Source	UC-01	Extend	Occurs only when a threat verdict is raised and --block policy is satisfied.

The documentation below defines the implemented preconditions, normal outcome and exceptions for the principal use cases. Authentication is not claimed because the local dashboard has no login mechanism, and active defence is described as optional and policy gated, matching the default configuration and current dispatcher behaviour.

Table 3.2: Principal MANTIS use case documentation.

ID and goal	Actor	Preconditions	Main success path	Alternative and postcondition
UC-01 Live monitoring	Security Administrator	Supported Linux host; sensor configured; raw capture permission for live mode.	Start sensor; capture frames; parse supported protocols; analyse relevant features; publish qualifying alerts.	Malformed frames are rejected without stopping the loop. Monitoring continues until a stop request; brain state and report are then written.

ID and goal	Actor	Preconditions	Main success path	Alternative and postcondition
UC-02 Investigate and export	Security Administrator	Web SIEM running and packet or alert records available.	Open dashboard; filter packets or alerts; inspect source, engine, severity and ATT&CK context; request JSON or PCAP evidence.	An empty store returns a valid empty state. Exported records follow the configured privacy and snap length controls.
UC-03 Train and restore models	Security Administrator	Labelled CSV fields match the selected loader; output directory is writable.	Validate rows; train the selected engines; serialise a timestamped JSON brain; import the latest compatible sections on start up.	Missing columns raise a dataset error. Missing or incompatible sections retain safe seeded state rather than corrupting the engine.
UC-04 Apply active defence	Security Administrator and Host OS	A threat identifies a source; blocking is explicitly enabled; kind is permitted; source is not whitelisted.	Dispatcher evaluates policy; ActiveDefense inserts one iptables DROP rule; response status and rule are recorded.	Disabled policy, failed command, duplicate source or whitelist match produces no block. MANTIS rules are removable during shutdown.

3.2 Sequence Diagrams

A sequence diagram shows messages in time order. During live detection, the Linux kernel supplies an Ethernet frame to the sniffer thread, which places raw bytes in the bounded queue. The analyser thread removes the frame, PacketParser validates and decodes it, and the relevant engines return Verdict objects. AlertDispatcher then applies one common route for logging, UDP/JSON telemetry, dashboard publication and optional response.

The response message occurs after the dispatcher has checked block enabled status, the configured block on categories and the whitelist. Brain export is a shutdown operation rather than part of every packet path. This ordering keeps the high frequency capture loop small and makes privacy processing occur at the persistence boundary.

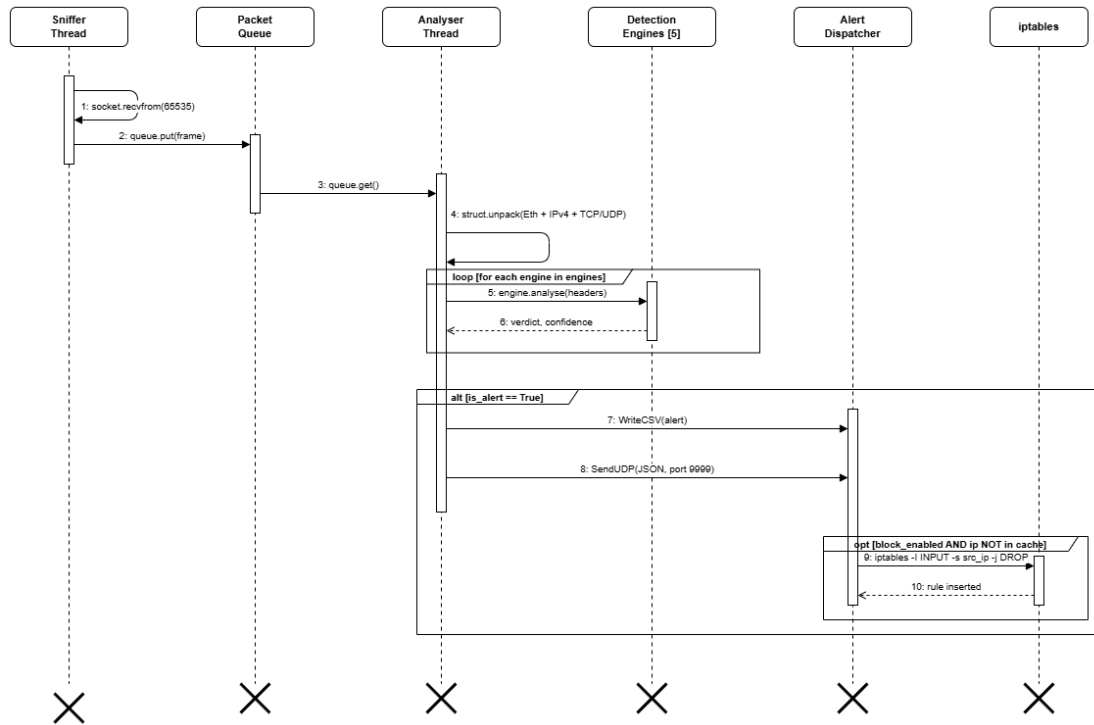


Figure 3.2a: Live detection sequence from captured frame to operator visible alert.

The training sequence shows the administrator invoking train.py, Loader validating labelled CSV records, the applicable engines fitting their state, and the completed brain snapshot being written as JSON.

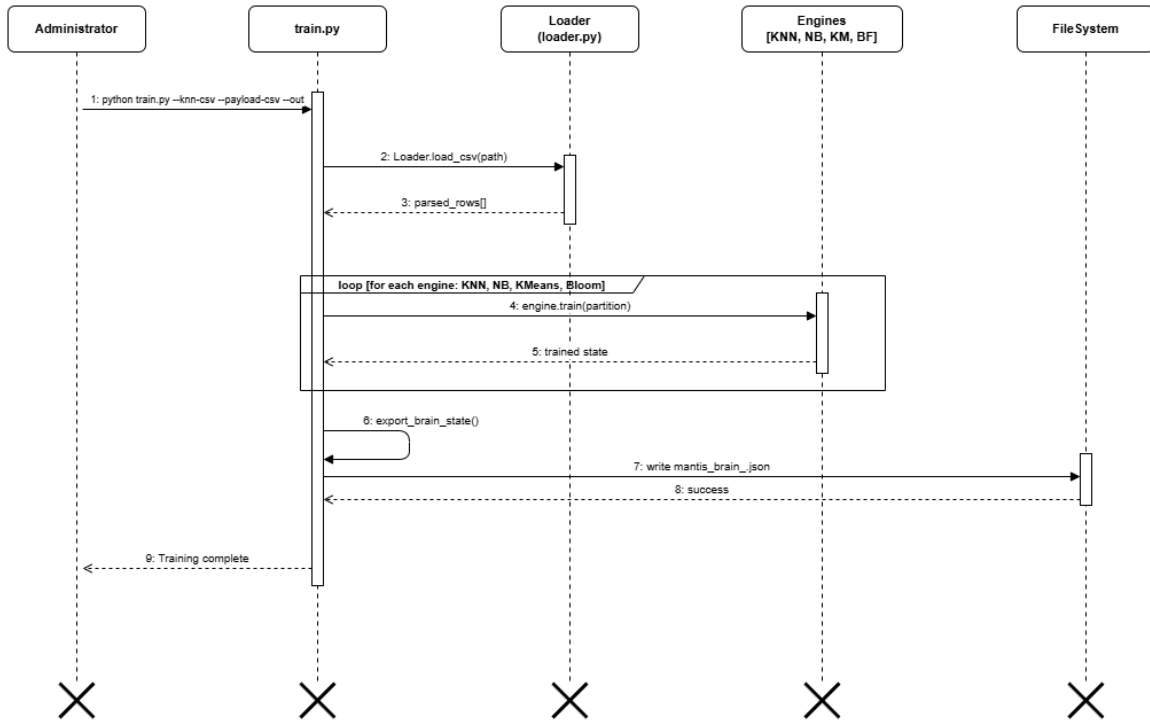


Figure 3.2b: Training pipeline from validated data to persistent engine state.

The SIEM sequence separates asynchronous alert ingestion from browser retrieval. AlertDispatcher sends JSON to the UDP listener, the listener updates the bounded alert buffer, and the browser obtains the current operator view over HTTP.

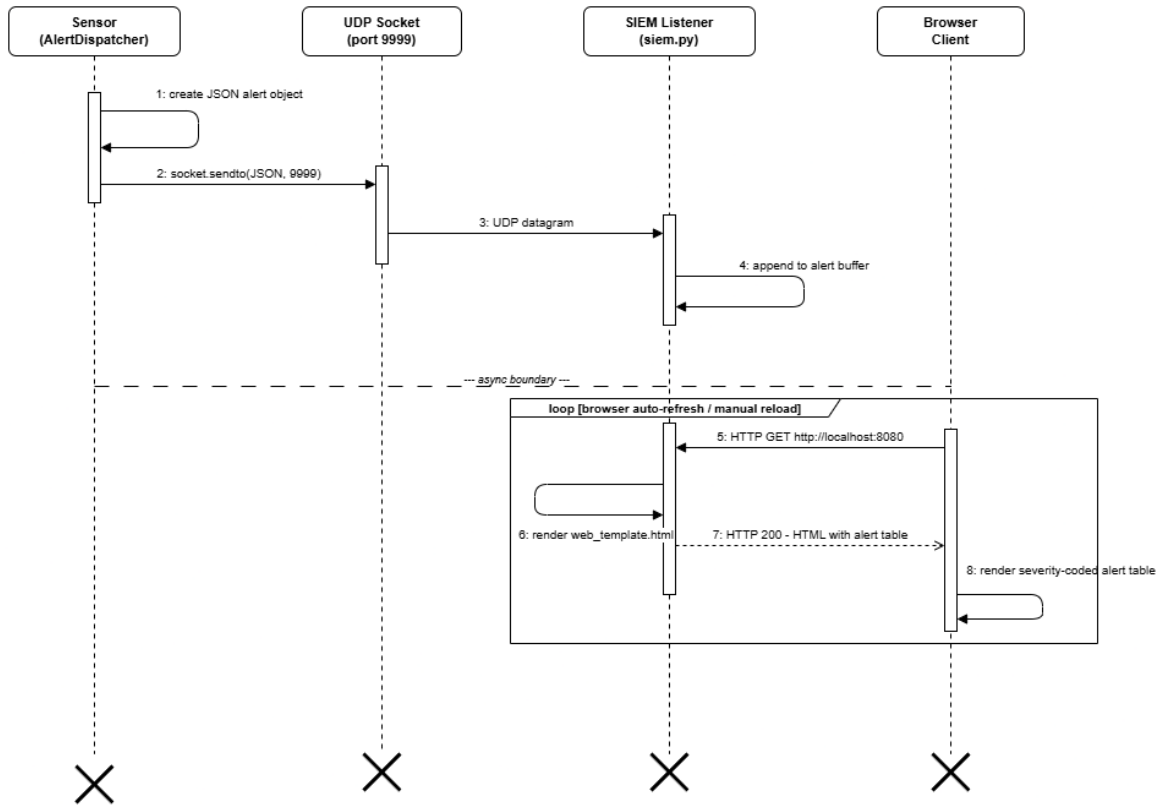


Figure 3.2c: SIEM alert delivery and browser dashboard retrieval.

The offline sequence provides a deterministic alternative to privileged live capture. main.py opens a finite PCAP stream, submits each decoded packet to the detection engines, and returns the completed analysis evidence to the administrator.

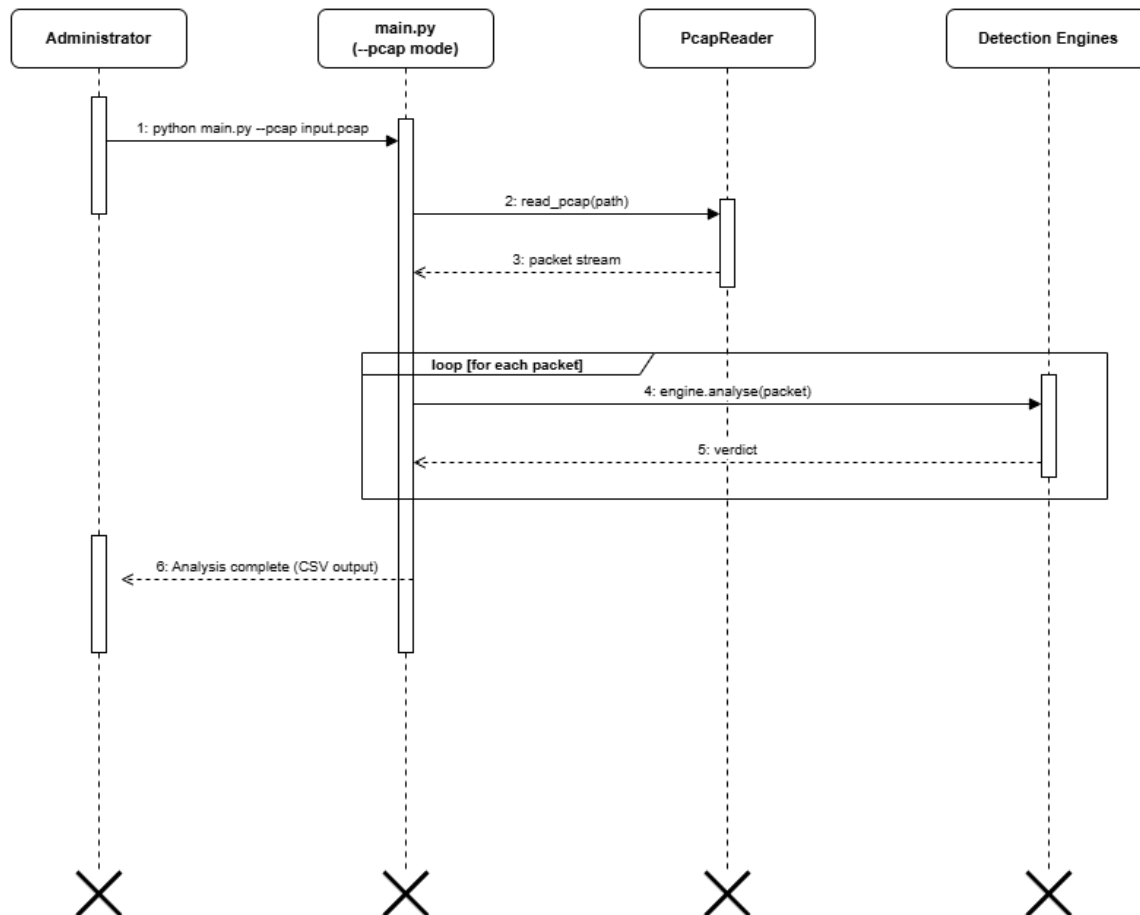


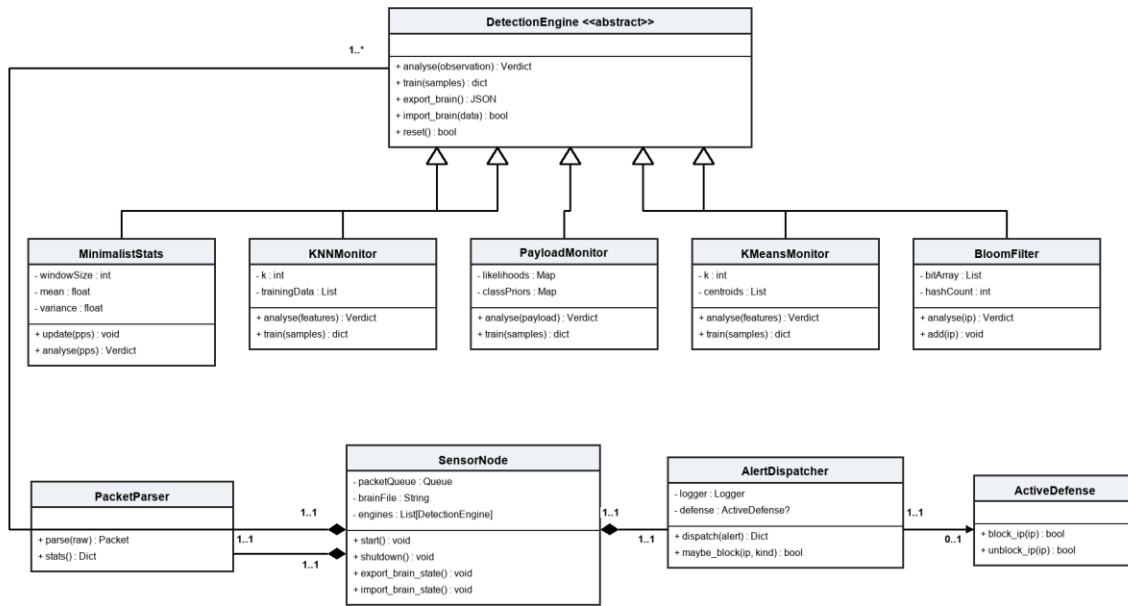
Figure 3.2d: Offline PCAP replay and finite analysis completion.

3.3 Class Diagram

The corrected class diagram uses standard multiplicity notation. One SensorNode (1..1) composes one or more DetectionEngine objects (1..*), exactly one PacketParser (1..1), and exactly one AlertDispatcher (1..1). AlertDispatcher can invoke zero or one ActiveDefense object (0..1). The five hollow triangle arrows, including BloomFilter, denote inheritance and therefore do not carry multiplicities.

The class diagram assigns static responsibility. SensorNode owns the capture lifecycle and composes PacketParser, AlertDispatcher and the five concrete engines. DetectionEngine defines the shared analyse, train, export_brain, import_brain and reset operations, while Verdict provides a consistent label, confidence, rationale and threat flag. This interface allows the dispatcher to consume results without depending on each algorithm's internal mathematics.

Filled diamonds identify parts whose lifecycle is controlled by SensorNode; the navigable arrow to ActiveDefense shows an optional service rather than ownership. The corrected notation separates relationship type from cardinality, so 1..1 means exactly one and 1..* means one or more. This matches SensorNode construction in sensor.py and the optional defense argument accepted by AlertDispatcher.



UML notation: hollow triangle = inheritance filled diamond = composition open arrow = navigable association 1..1 exactly one 1..* one or more 0..1 optional

Figure 3.3: Corrected MANTIS class model with inheritance, composition and explicit UML multiplicities.

3.4 Activity Diagrams

Live detection moves each captured frame through the bounded queue, parser and five engine fork. Verdicts are joined before alert persistence, SIEM delivery and policy controlled response, while normal traffic returns without a response action.

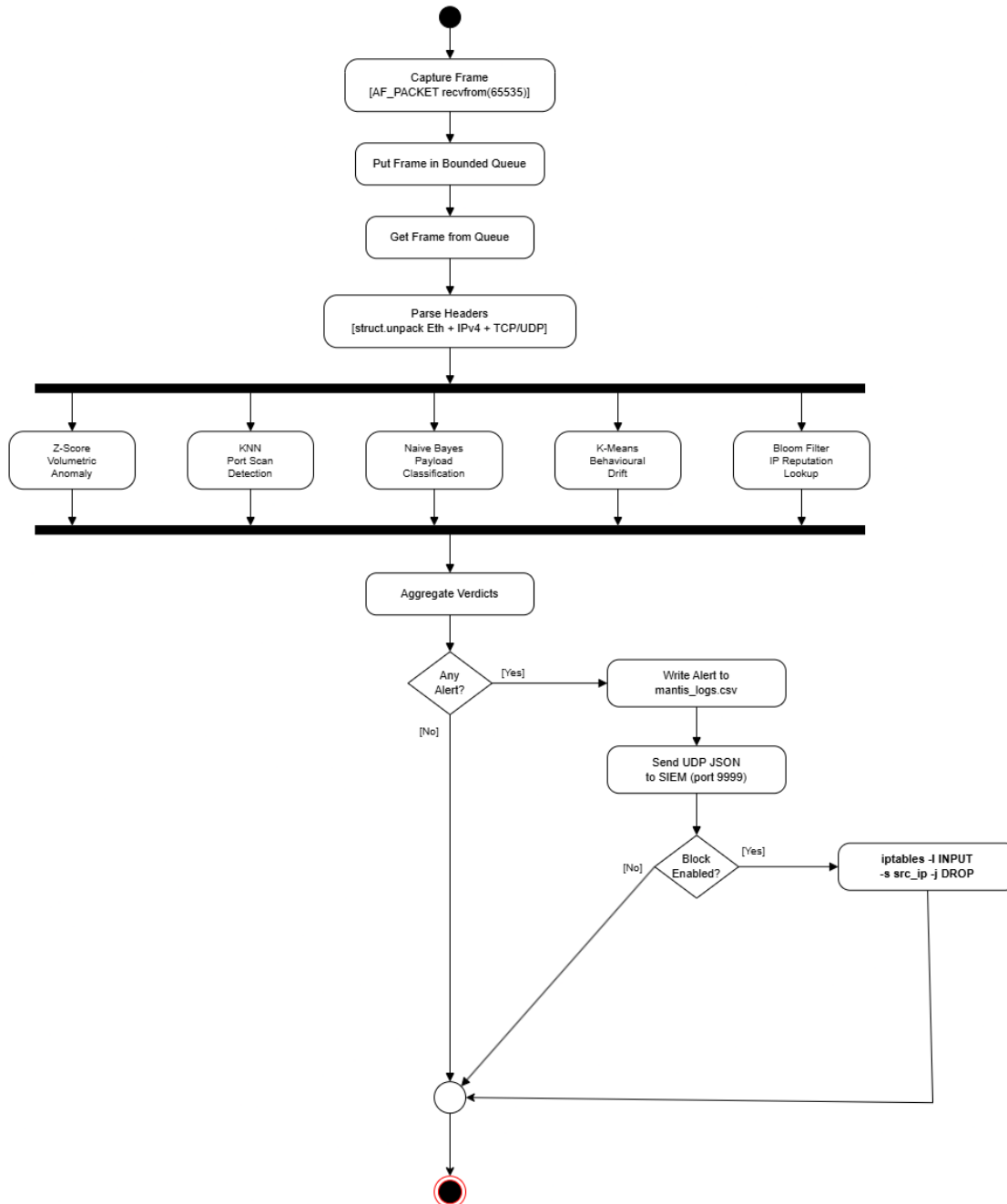


Figure 3.4a: Live detection, verdict aggregation and optional response activity.

Training validates and projects the supplied data before fitting KNN, Naive Bayes and K-Means and seeding the Bloom Filter. The parallel results converge into one JSON brain export and a training summary.

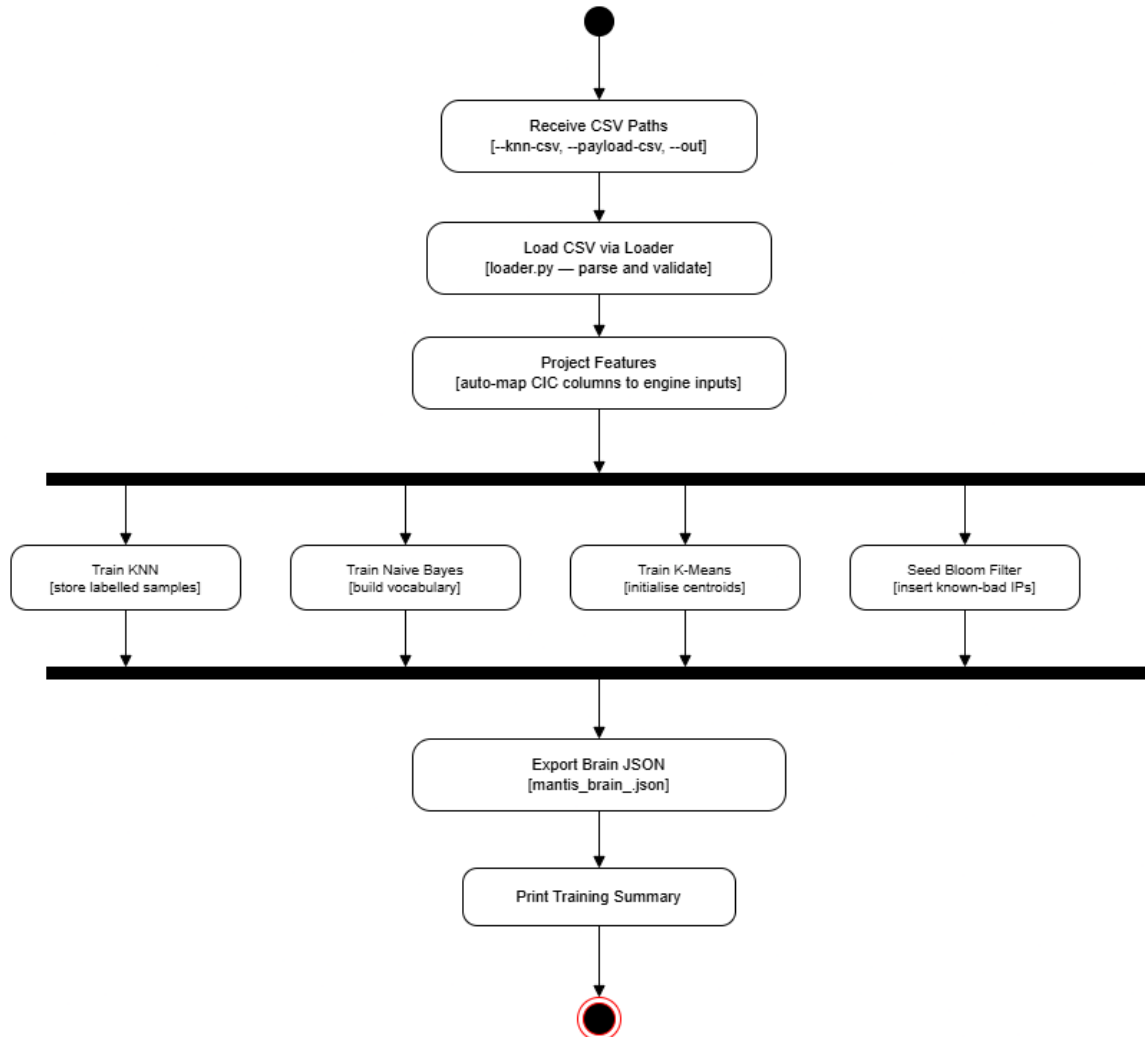


Figure 3.4b: Training pipeline from input validation to brain export.

Evaluation imports trained state, classifies every labelled sample and updates TP, FP, FN and TN counts. After the final sample, the harness calculates precision, recall, F1 and accuracy for reproducible interpretation.

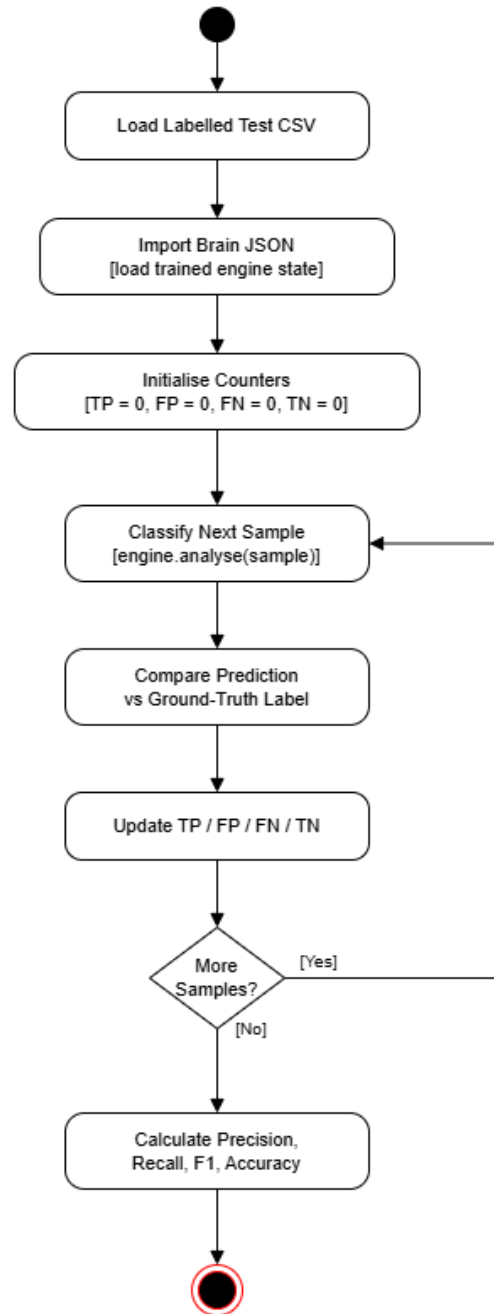


Figure 3.4c: Evaluation harness and confusion matrix metric activity.

An activity diagram represents processing steps, guarded alternatives and end states. The corrected brain persistence flow begins with engine initialisation and a search for the newest timestamped

JSON file. Only after this work does the system decide whether compatible learned state is available. A successful import and a safe default path both converge on the ready state before monitoring starts.

A stop request leaves the monitoring activity, closes threads and any PCAP writer, exports all engine states, writes the timestamped brain file and generates the forensic report. The first node after Start is therefore an action, not a startup or shutdown decision. This matches the SensorNode lifecycle implemented by `import_brain_state()`, `start()`, `shutdown()` and `export_brain_state()`.

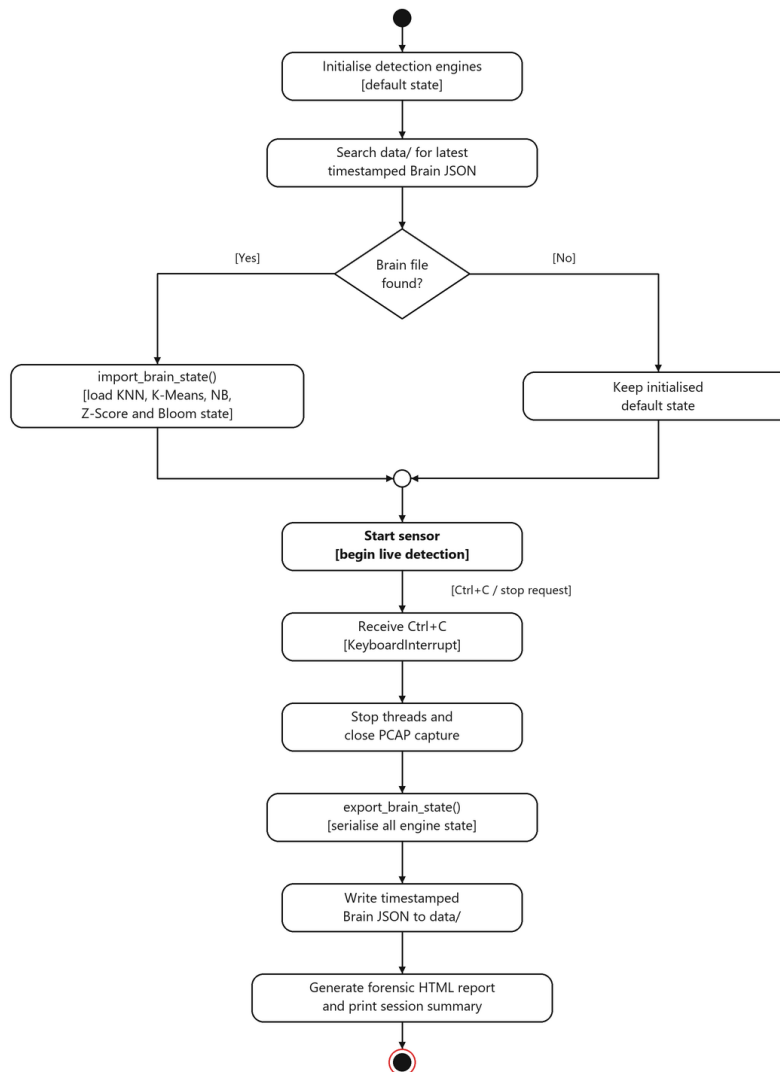


Figure 3.4d: Corrected brain persistence and orderly shutdown activity flow.

Installation identifies the Linux package manager, installs the operating system firewall facility, grants the required capture capabilities and creates the command entry point before reporting success.

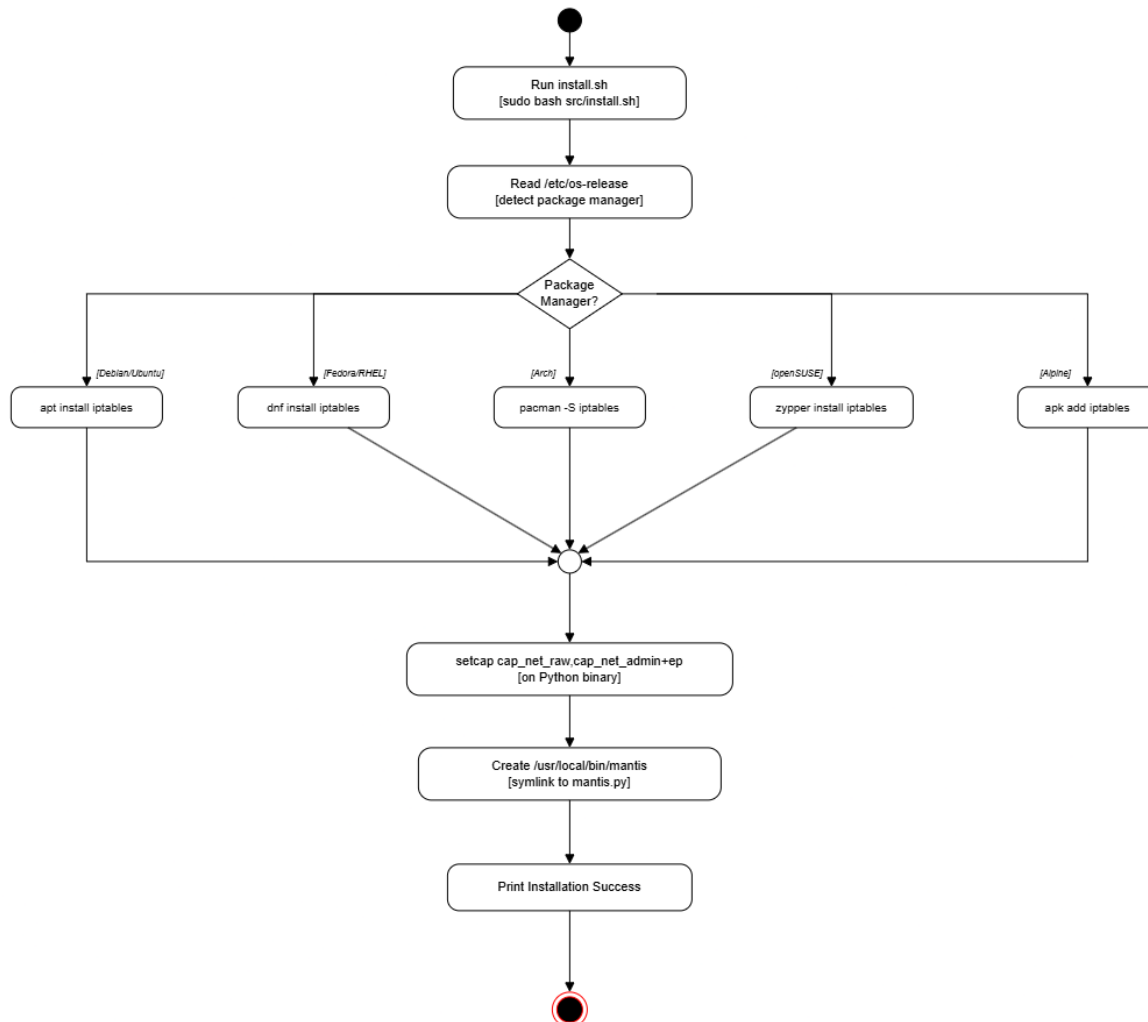


Figure 3.4e: Linux installation and capability configuration activity.

3.5 Database Schema

MANTIS does not use a relational database. The zero dependency requirement favours bounded memory and human readable files, so this section documents a logical data schema and its physical flat file mappings. Packet events may produce multiple engine verdicts; a threatening verdict may produce one alert; and an alert may produce one firewall rule only when response policy permits it. Timestamps, source addresses and engine details provide traceability across these records.

Alerts are appended to CSV after redaction, UDP telemetry is encoded as JSON, engine state is stored in timestamped JSON brain files, packet evidence may be exported as PCAP, and session findings may be rendered as HTML. The packet store is a bounded in memory deque rather than durable history. This design improves portability and inspection, but it does not provide relational querying, concurrent transactions or long term enterprise retention.

Table 3.3: Logical to physical persistence mapping.

Logical record	Physical representation	Principal fields	Purpose
Packet event	In memory PacketStore; optional PCAP	Timestamp, addressing, protocol, ports, length, frame excerpt	Bounded investigation and replay evidence
Threat alert	CSV and UDP/JSON	Engine, message, confidence, MITRE fields, severity, risk, sanitised excerpt	Audit trail and real time SIEM ingestion
Brain snapshot	mantis_brain_*.json	KNN samples, K-Means centroids, NB likelihoods, Z-Score statistics, Bloom bits	Readable restart and training state
Firewall response	Runtime iptables rule and dispatcher summary	Source, policy category, applied status	Opt in and reversible local containment
Forensic report	HTML	Session summary and alert chronology	Human readable hand off evidence

3.6 Algorithms Implemented

Chapter 2 introduced the five detection algorithms. This section completes the design account for the two stateful engines not expanded there: online Mini-Batch K-Means for distance based novelty and a Bloom Filter for reputation membership. Both implement the common engine contract and serialise their state into the JSON brain.

3.6.1 Online Mini-Batch K-Means Engine

KMeansMonitor receives the input feature vector $f = (\text{packets per second}, \text{unique ports})$ and represents normal traffic with k centroids. It selects the nearest centroid c by Euclidean distance and returns an anomaly only when the distance exceeds the configured novelty threshold. A normal observation updates the centroid incrementally as $c_{\text{new}} = c_{\text{old}} + \alpha(f - c_{\text{old}})$, allowing bounded online adaptation without retaining the full stream.

Mathematical Model

For nearest centroid c_k : $c_k := c_k + \alpha * (x - c_k)$ with learning rate α , following the mini batch update of Sculley (2010).

Step by Step Procedure

1. Compute distance from point x to each centroid.
2. Assign x to argmin centroid index.
3. Update centroid with learning rate α .
4. Flag alert if distance exceeds configured threshold.

Pseudocode

```
Mini-Batch K-Means classify ( $x = [\text{pps}, \text{unique\_ports}]$ ):  
    nearest_k = argmin (euclidean ( $x, c$ ) FOR  $c$  IN centroids)  
    min_dist = euclidean ( $x, \text{centroids}[\text{nearest\_k}]$ )  
    centroids[nearest_k] +=  $\alpha * (x - \text{centroids}[\text{nearest\_k}])$  // Sculley 2010  
    IF min_dist > threshold: RETURN BehaviouralDrift  
    ELSE: RETURN Normal
```

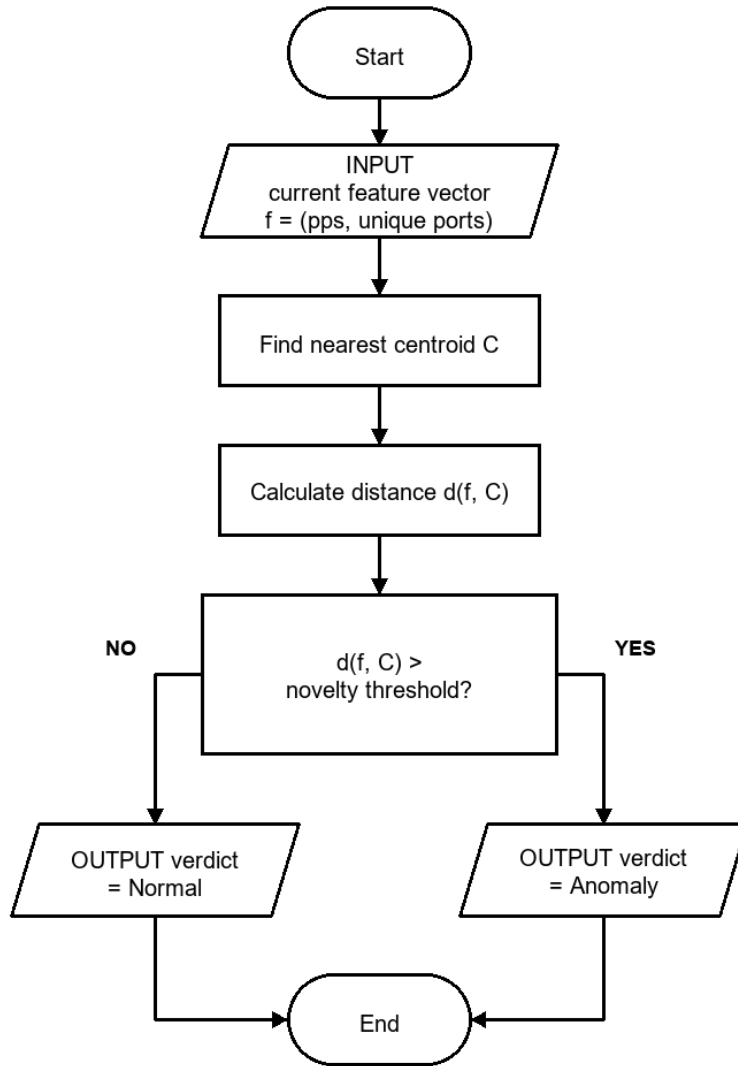


Figure 3.5: Online Mini-Batch K-Means flow from explicit feature input to normal or anomaly output.

3.6.2 Bloom Filter Reputation Engine

BloomFilter stores known malicious source indicators in a bit array. Adding an indicator computes k hash positions and sets their bits. A lookup returns Unknown when any required bit is zero and returns Blacklisted only when all bits are set. Lookup cost is $O(k)$, independent of the number of inserted indicators, which makes the reputation check suitable for the per packet path.

Mathematical Model

m bits and k hash functions give an expected false positive rate of $(1 - \exp(-kn/m))^k$ for n inserted items (Mitzenmacher and Upfal, 2017).

Step by Step Procedure

1. Hash item with k independent hash functions.
2. Add all k bit positions to 1.
3. On check: return true only if all k positions are 1.

Pseudocode

```
Bloom Filter check(ip):  
  FOR seed IN hash_seeds:  
    pos = adler32(encode(ip + str(seed))) mod m  
  IF bit_array[pos] == 0: RETURN Clean // Definite non member  
  // All k bits set -> probable member  
  fp_rate = (1 - exp(-k*n/m))^k          // Mitzenmacher & Upfal 2017  
  RETURN KnownMalicious, confidence = 1.0 fp_rate
```

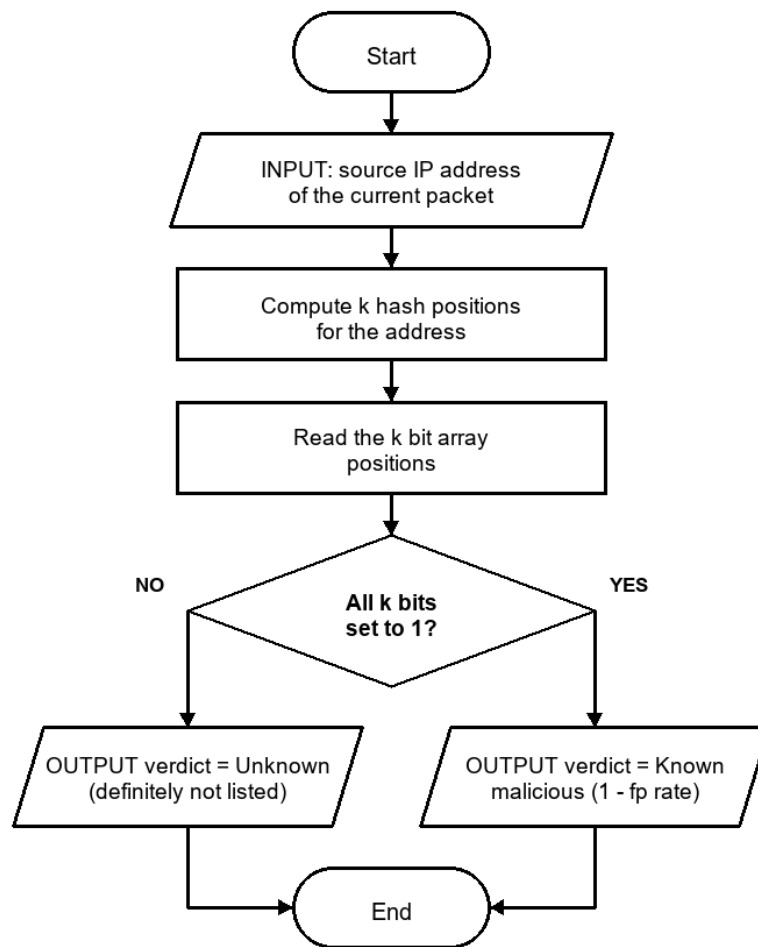


Figure 3.6: Bloom Filter reputation membership decision flow.

3.7 System Interface and Design

The system interface supports monitoring and investigation without requiring a separate SIEM dependency. The terminal view serves headless operation, while the embedded web console exposes the current packet rate, bounded packet history, alerts, risk and effective configuration. REST endpoints provide statistics, packet records, configuration and PCAP export; UDP port 9999 receives structured alerts and HTTP port 8080 serves the interface.

Interface decisions follow operator questions rather than source modules: what happened, where it originated, which engine decided, how severe it is and what evidence can be exported. Severity

always has a textual label, ATT&CK identifiers give investigation context, and empty states explain that monitoring is waiting for data. Active defence and privacy settings remain visible through configuration data rather than being implied by colour alone.

3.7.1 Wireframe

The low fidelity wireframes in Figures 3.7a to 3.8d validate the layout and information hierarchy of every implemented console page before colours and demonstration data are applied. The shared header preserves sensor state and navigation. Each page then answers a separate operator question: whether the host and detector are healthy, which packets were observed, which events require investigation, and how the detection pipeline produces and reports a verdict.

Overview page

The Overview wireframe prioritises host CPU, memory, online sensor count and current threat event count. Recent classified events occupy the largest region, while the engine panel separates

Z-Score, KNN, K-Means, Naive Bayes and Bloom Filter activity so an operator can distinguish an idle detector from a failed sensor.

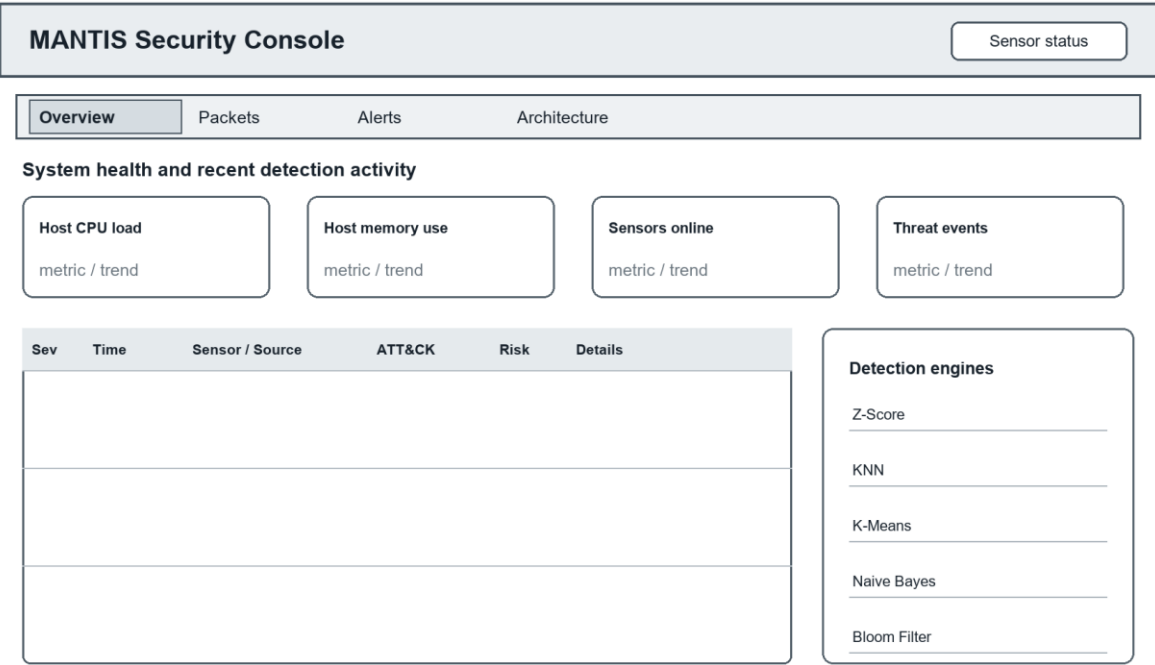


Figure 3.7a: Overview page wireframe showing system health, recent events and engine activity.

Packets page

The Packets wireframe places the search field, protocol filter, pause control, auto scroll state and PCAP or JSON export above the bounded capture table. Selecting a row opens decoded and

hexadecimal detail below, supporting independent verification without pretending that the browser is a durable forensic database.

MANTIS Security Console

Sensor status

Overview

Packets

Alerts

Architecture

Live packet capture

Search, protocol filter, pause, auto-scroll and evidence export

Filter by IP, port or details

Protocol

Pause

Auto-scroll

Export PCAP

Export JSON

No.	Time	Source	Destination	Protocol	Length	Info

Selected packet detail: decoded fields, hexadecimal and ASCII

Figure 3.7b: Packets page wireframe showing capture controls, packet history and selected frame detail.

Alerts page

The Alerts wireframe reserves the full content width for the current in memory threat log. Severity, time, sensor and source, ATT&CK technique, risk and message remain visible in one scan path. The live label and bounded event note make the freshness and retention limit explicit.

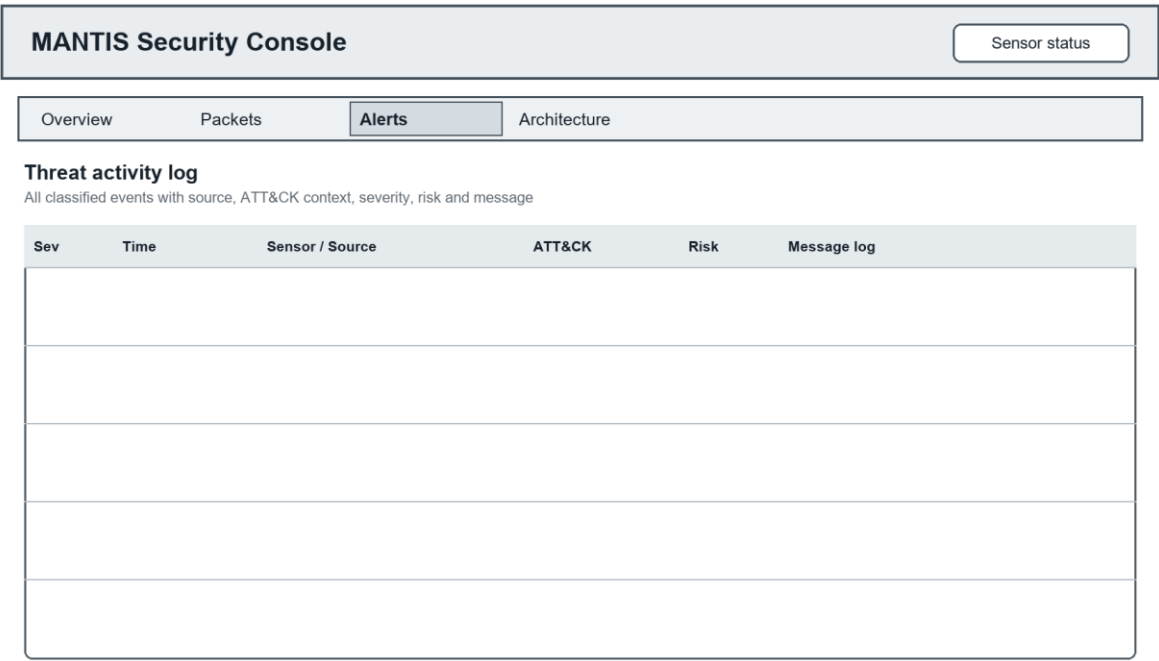


Figure 3.7c: Alerts page wireframe showing the complete bounded threat activity log.

Architecture page

The Architecture wireframe documents the five implemented engines separately and connects their explicit inputs and decisions to the common Verdict, response and reporting path. This page is

explanatory rather than interactive, allowing a reviewer or operator to understand what the system can and cannot infer.

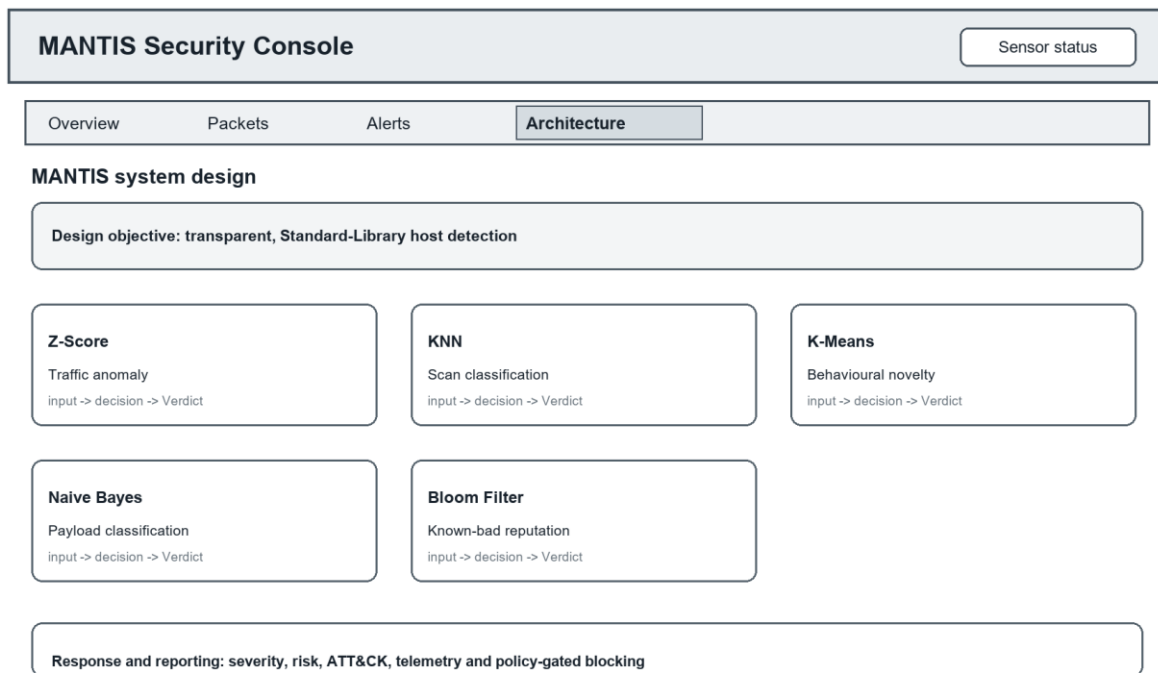


Figure 3.7d: Architecture page wireframe showing all five engines and the response path.

3.7.2 Mockup

The implemented function mockups in Figures 3.8a to 3.9d apply the current console styling and API contract to the same four pages. Values and events are controlled demonstration records, not production incidents. The mockups include only behaviour supported by `web_template.html` and `siem.py`, so the visual design remains traceable to the running prototype rather than presenting aspirational screens.

Overview page

The realised Overview page combines host health cards, recent classified events and counts for every detection engine. Text labels accompany severity and system state. This permits rapid triage while retaining the engine and ATT&CK context needed to explain why an event appears.

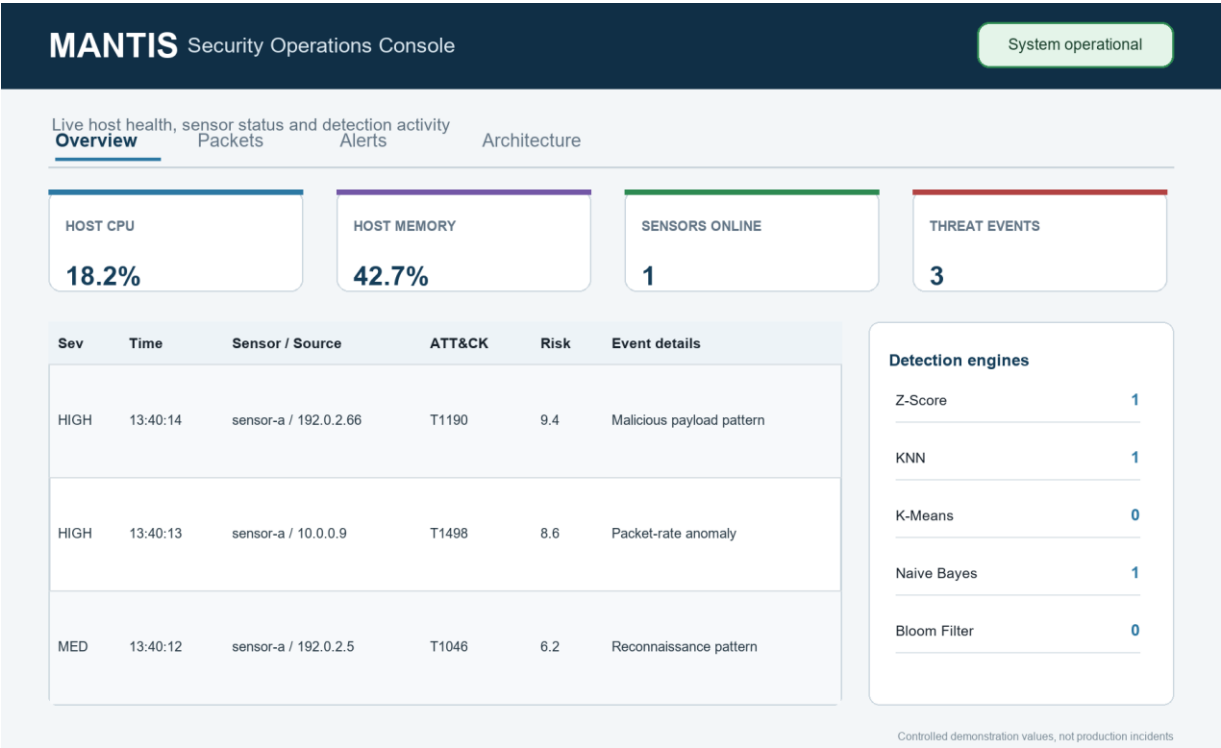


Figure 3.8a: Overview page mockup with controlled health metrics, recent events and five engine activity.

Packets page

The realised Packets page mirrors the implemented raw socket workflow. Search and protocol controls reduce the visible buffer, pause and auto scroll manage observation, PCAP and JSON

actions export evidence, and a selected row reveals decoded fields plus hexadecimal content for independent inspection.

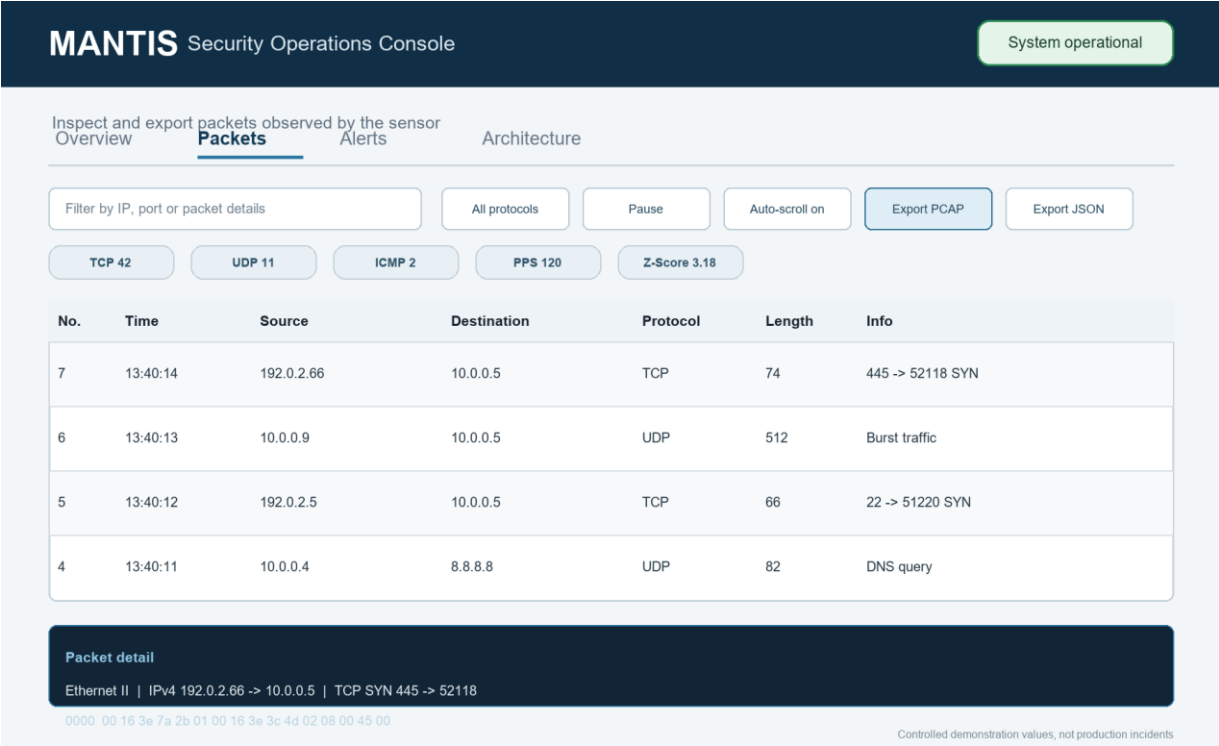
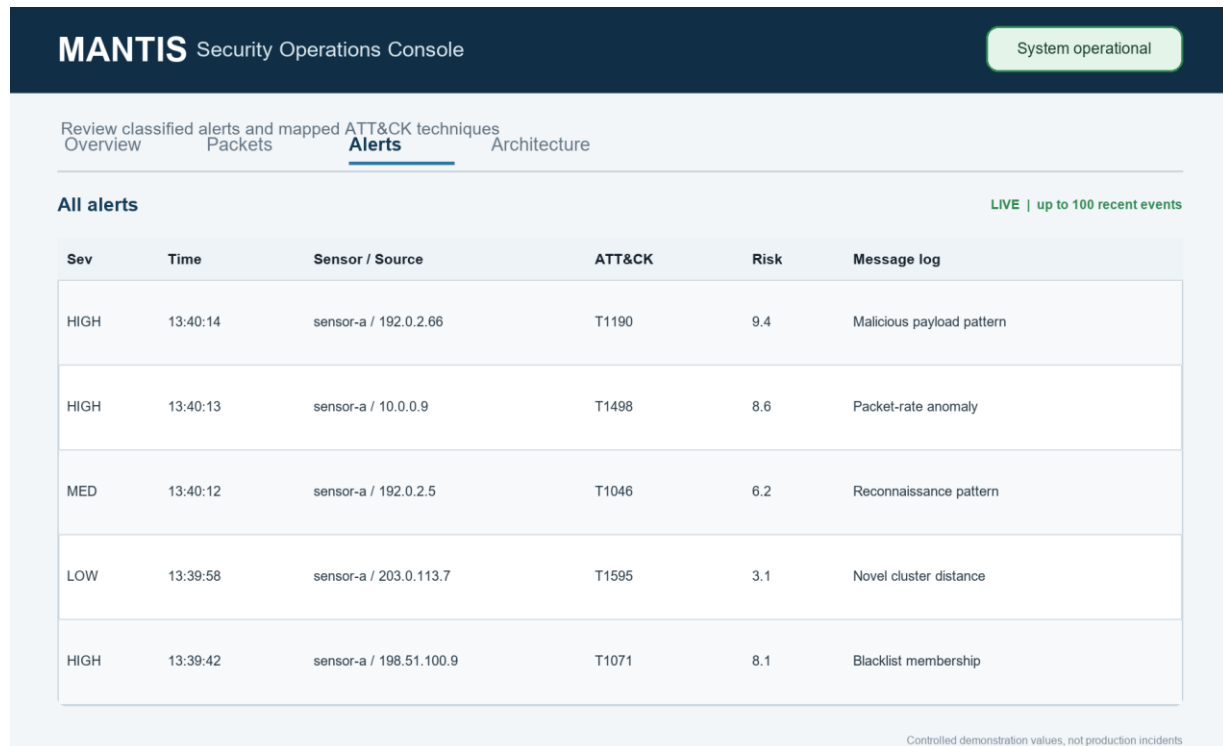


Figure 3.8b: Packets page mockup with filters, protocol counters, exports and packet detail.

Alerts page

The realised Alerts page presents up to 100 recent events in descending time order. Each row preserves the textual severity, sensor and source relationship, ATT&CK mapping, numeric risk

and message detail. The design avoids inventing acknowledgement, assignment or case management controls that the embedded server does not implement.



The mockup shows the MANTIS Security Operations Console. At the top, a dark blue header contains the title 'MANTIS Security Operations Console' and a green 'System operational' status button. Below the header, a navigation bar includes links for 'Overview', 'Packets', 'Alerts' (which is underlined), and 'Architecture'. The main content area is titled 'All alerts' and includes a 'LIVE | up to 100 recent events' indicator. A table displays five alert entries with columns for severity, time, sensor/source, ATT&CK technique, risk score, and message log. The table is followed by a disclaimer: 'Controlled demonstration values, not production incidents'.

Sev	Time	Sensor / Source	ATT&CK	Risk	Message log
HIGH	13:40:14	sensor-a / 192.0.2.66	T1190	9.4	Malicious payload pattern
HIGH	13:40:13	sensor-a / 10.0.0.9	T1498	8.6	Packet-rate anomaly
MED	13:40:12	sensor-a / 192.0.2.5	T1046	6.2	Reconnaissance pattern
LOW	13:39:58	sensor-a / 203.0.113.7	T1595	3.1	Novel cluster distance
HIGH	13:39:42	sensor-a / 198.51.100.9	T1071	8.1	Blacklist membership

Figure 3.8c: Alerts page mockup with the complete in memory threat log and ATT&CK context.

Architecture page

The realised Architecture page documents Z-Score, KNN, K-Means, Naive Bayes and Bloom Filter as five distinct methods, then shows the common response path from Verdict to risk,

evidence, telemetry, dashboard and optional policy gated blocking. It corrects the risk of hiding separate engines behind one generic machine learning label.

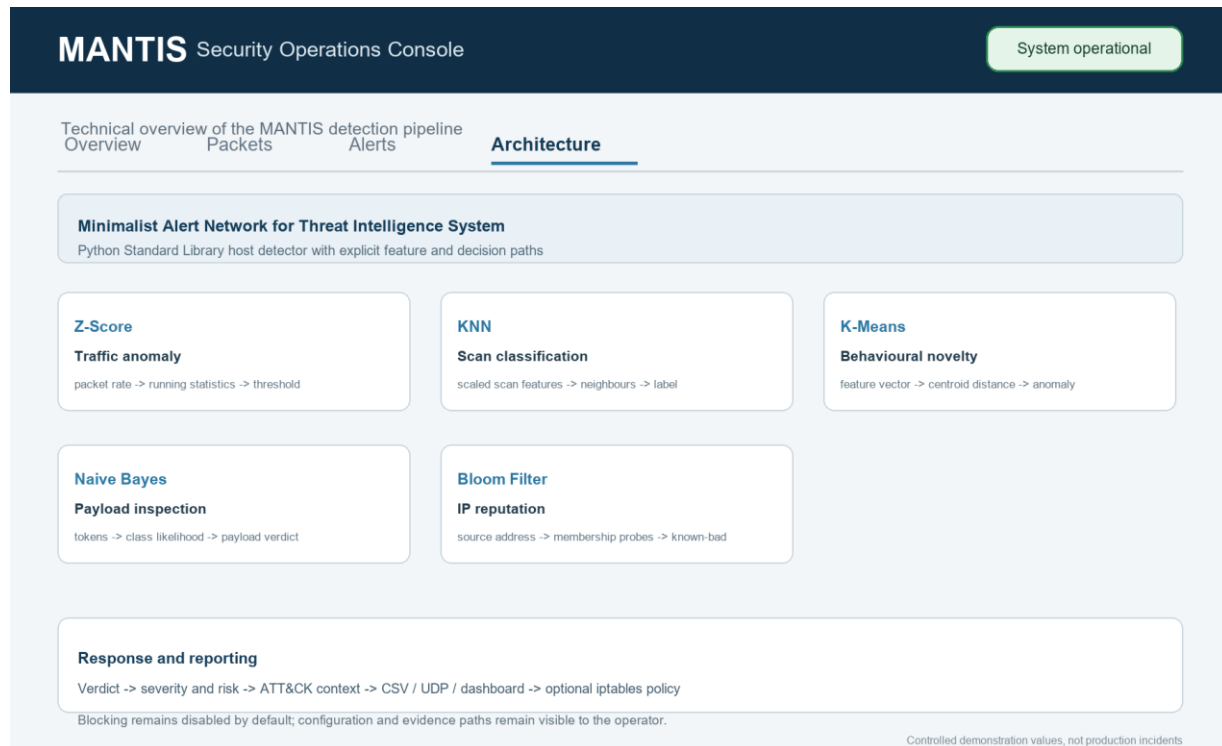


Figure 3.8d: Architecture page mockup documenting all five engines and policy gated response.

The console also supports responsive layout, protocol filtering, packet details, effective policy reporting and bounded evidence export. Authentication, TLS, multi user case management and durable database search are intentionally absent from the mockups because they require a separate security and persistence design beyond the embedded local prototype.

4. System Build and Technical Notes

The approved design was built as a Python Standard Library application. The executable path begins in main.py, where command line options are merged with environment, JSON file and seeded defaults. Live operation checks for Linux AF_PACKET capability, starts the optional web SIEM, constructs SensorNode and launches separate capture and analysis threads. Offline PCAP replay exercises the same parser and engines without raw socket privileges.

Packet Capture and Analysis Build

SensorNode reads Ethernet frames through a SOCK_RAW AF_PACKET socket and places them in a bounded queue. PacketParser validates Ethernet, optional 802.1Q, IPv4, TCP, UDP and ICMP structure using explicit binary unpacking. Truncated or unsupported frames return no event rather than terminating the analyser. Per second statistics supply the Z-Score, KNN and K-Means engines, while payload bytes and source addresses are routed only to the engines that require them.

Detection, Dispatch and Response Build

Each engine returns a Verdict containing label, confidence, detail and is_threat. AlertDispatcher enriches a qualifying result with ATT&CK context, writes the CSV audit record, sends JSON telemetry, updates the attached interface and then evaluates the optional blocking policy. ActiveDefense issues an iptables DROP command only when blocking is enabled, the detection category is permitted and the source is not whitelisted; duplicate sources are not inserted repeatedly.

Training, State and Evidence Build

Dataset loaders validate the columns required by KNN, Naive Bayes and K-Means and reject a CIC export that incorrectly substitutes destination port for an aggregated unique port count. Training writes a human readable brain document, and SensorNode restores compatible sections from the newest file. Logger, UDPSender, PCAP utilities, PacketStore and ReportGenerator provide complementary evidence paths, with privacy redaction applied before material leaves volatile analysis state.

Technical Constraints

Live capture and iptables response are Linux specific and require suitable capabilities or administrative permission. Payload analysis cannot inspect encrypted application content, UDP telemetry is intentionally lightweight rather than guaranteed delivery, and the in memory packet store is bounded rather than a forensic database. The embedded HTTP server is suitable for a controlled local prototype, not an internet facing multi user deployment. These constraints define the conditions under which the build should be evaluated.

Table 4.1: MANTIS build components and technical constraints.

Build area	Principal modules	Implemented responsibility	Technical note
Entry and configuration	main.py, config.py	Argument parsing, precedence, safe defaults, privilege check	No third party runtime package
Acquisition and parsing	sensor.py, parser.py	AF_PACKET producer, bounded queue, protocol decoding	Live capture is Linux specific
Detection	engines/*.py	Five engines under DetectionEngine and Verdict	State exported through a uniform JSON contract
Dispatch and response	dispatcher.py, active_defense.py	Audit, telemetry, UI publication and policy gated iptables	Blocking is off by default
Presentation and evidence	siem.py, web_template.html, pcap.py, reporting.py	HTTP/UDP console, packet export and HTML report	HTTP 8080 and UDP 9999 by default
Training and evaluation	training/*, evaluation/*, replay.py	Validated loaders, model training, metrics and deterministic replay	External datasets must match engine semantics

5. Testing Strategy and Evaluation

Testing establishes whether the implementation behaves as specified; evaluation interprets whether the observed behaviour supports the dissertation's claims. MANTIS separates component tests, integration tests and system tests so that a correct formula is not mistaken for a correct end to end sensor. Algorithm metrics and replay evidence are interpreted separately from software test pass counts.

The current repository was re executed on 20 August 2026 using unittest discovery. It ran 116 tests in 4.642 seconds and all tests passed. A small number of ResourceWarning messages reported test process UDP sockets that were not explicitly closed; they did not fail any assertion but remain a test harness clean up item. No new CPU, memory or latency result is asserted here because those measurements require a declared reference environment and controlled workload.

5.1 Testing Definitions and Approach

Unit tests isolate one rule. Integration tests cross real module boundaries without requiring root. System tests bind a real UDP socket and HTTP server on loopback. Regression tests lock fixes for flood/scan confusion, baseline poisoning and stealth scans. Table 5.1 maps these levels to FR/NFR identifiers and the unittest modules that were executed.

Table 5.1: Test levels mapped to requirements and evidence modules.

Test level	FR / NFR traced	What is exercised	Evidence module
Unit	FR4, FR12, NFR2, NFR8	Engine mathematics, config, privacy, PCAP helpers	test_engines.py, test_modules.py
Integration	FR1–FR8, FR10, FR14–FR22, NFR3, NFR4	Parser→engines, dispatcher, brain, replay, packet store	test_integration.py
System	FR5, FR9, FR18, NFR7	Real UDP listener and HTTP SIEM on loopback	test_siem_api.py

Test level	FR / NFR traced	What is exercised	Evidence module
Regression	FR4, NFR2	Flood not scan, baseline poisoning, stealth scan, K- Means non poisoning	test_modules.py / test_engines.py
Training	FR5, FR6, FR12	Column validation and brain serialisation	test_training_and_eval.py

The approach combines positive cases, negative cases, boundary conditions and failure handling. Expected attacks must be detected, normal traffic must remain normal, malformed input must not crash the pipeline, privacy sensitive values must not reach persistence, and active defence must remain disabled unless every policy condition is satisfied. Synthetic frames and a generated PCAP make the pipeline reproducible without attacking an external network.

5.2 Algorithm Tests

Engine tests check both detection and non detection behaviour. Z-Score verifies warm up, Welford stability, sustained flood detection and baseline poisoning protection. KNN verifies scaled features, normal browsing, flood versus scan separation and low volume scans. K-Means verifies warm up, familiar traffic, novelty distance and withholding anomalous updates. Naive Bayes tests benign payloads, malicious tokens and the single token ADMIN safeguard. Bloom tests feed loading, membership, saturation and the no false negative property for inserted indicators.

The saved domain matched evaluation artefact reports the results below. These are small verification sets aligned with each engine's intended input, so the results demonstrate implementation behaviour rather than general field accuracy. Naive Bayes missed one of twelve malicious payloads; the other engines achieved perfect scores on their small samples. The result should not be compared directly across attack types because the datasets and decision problems differ.

Table 5.2: Small domain matched algorithm verification results.

Engine	Samples	Precision	Recall	F1	MCC
KNN port scan	40	1.0000	1.0000	1.0000	1.0000
Naive Bayes payload	24	1.0000	0.9167	0.9565	0.9199
Z-Score flood	40	1.0000	1.0000	1.0000	1.0000
K-Means novelty	44	1.0000	1.0000	1.0000	1.0000
Bloom reputation	25	1.0000	1.0000	1.0000	1.0000

5.3 Test Cases and Scenarios

The selected cases below trace the main functional and safety requirements to observable outcomes. The generated replay contains a benign warm up followed by port scan, flood, malicious payload and known bad source phases. System tests use loopback services and controlled datagrams, while firewall tests substitute a recording defence object so the suite never modifies the host firewall.

Table 5.3 lists expected against actual outcomes: every listed case passed in the 20 August 2026 run (116 tests, 4.642 s, unittest discovery), and the actual results matched the expected column. Each case was then re executed in isolation on the same day and the terminal output was captured; this evidence is reproduced case by case in Table 5.4. The results verify the implementation and do not certify production accuracy on unseen operational traffic.

Table 5.3: Twenty eight representative executed test cases and scenarios.

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-01	Unit	Benign payload	Return Safe with no threat	Pass	test_engines.TestNaiveBayes.test_normal_http_is_safe

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-02	Unit	UNION SELECT payload	Return malicious payload verdict with keywords	Pass	test_engines.TestNaiveBayes.test_sqli_is_malicious
TC-03	Regression	High volume, one port input to KNN	Classified Normal, not Scan	Pass	test_modules.TestKNNFeatureScaling.test_flood_is_not_reported_as_a_port_scan
TC-04	Regression	Sustained flood after Z-Score warm up	Repeated alerts without baseline poisoning	Pass	test_modules.TestZScore / test_engines.TestZScore.test_spike_triggers_high_z
TC-05	Unit	Inserted and unknown Bloom indicators	Inserted found; unknown clean in controlled set	Pass	test_engines.TestBloom.test_added_item_found
TC-06	Integration	Malformed and truncated frames	Reject safely; analyser remains available	Pass	test_integration.TestParserToEngines.test_truncated_frame_does_not_raise

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-07	Integration	Credential and email in alert	Sensitive values absent from CSV and UDP	Pass	test_integration.TestDispatcherFanOut.test_pii_is_redacted_before_persistence
TC-08	Integration	Disabled, enabled and whitelist policy	Only enabled, non whitelisted case blocks	Pass	test_integration.TestDispatcherFanOut.test_blocking_disabled_by_default
TC-09	System	UDP alert to SIEM and HTTP API query	Alert includes technique, severity and risk	Pass	test_siem_api / test_integration.dispatcher_udp_path
TC-10	System	Buffered frame PCAP export	Valid PCAP reopens; every frame decodes	Pass	test_integration.TestPacketStoreToPcap.test_buffer_exports_as_readable_pcap
TC-11	Replay	13,159 frame five phase replay	No baseline alert; engines fire in intended phases	Pass	test_integration.TestReplayPipeline.test_every_engine_detects_its_own_attack

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-12	Persistence	Five engine brain round trip	Restored state matches export	Pass	test_integration.TestBrainRoundTripAcrossEngines.test_all_five_engines_round_trip
TC-13	Unit	K-Means warm up	No novelty alert before sufficient baseline observations	Pass	test_modules.TestKMeansNovelty.test_no_alert_during_warm_up
TC-14	Regression	K-Means anomalous observation	Novelty is flagged and does not poison centroids	Pass	test_modules.TestKMeansNovelty.test_anomalies_do_not_poison_the_model
TC-15	Regression	Low volume stealth scan	KNN detects scan after feature scaling	Pass	test_modules.TestKNNFeatureScaling.test_low_volume_stealth_scan_is_detected
TC-16	Unit	Invalid configuration value	Bad override is ignored without startup failure	Pass	test_modules.TestConfig.test_bad_env_value_does_not_crash

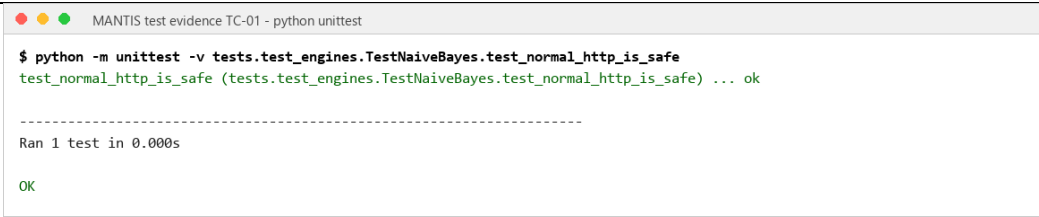
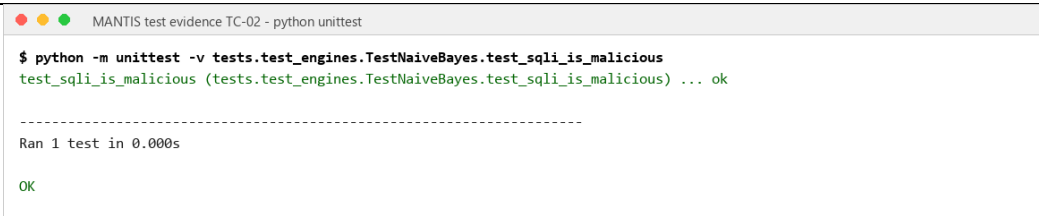
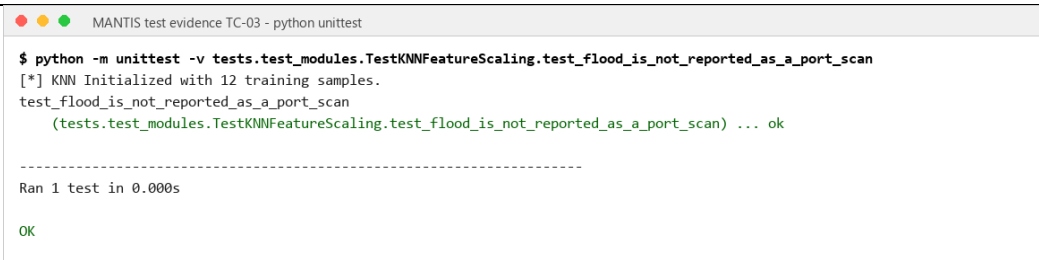
ID	Level	Scenario	Expected outcome	Status	Evidence
TC-17	Unit	Threat intelligence feed	Comments are stripped and inserted indicators have no false negatives	Pass	test_modules. TestBloomThreatIntel.test_no_false_negatives
TC-18	Integration	VLAN and ICMP frames	Supported frames decode into normalised packet records	Pass	test_integration. TestParserToEngines.test_vlan_tagged_frame_is_decoded
TC-19	Integration	Packet store capacity	Old entries rotate and memory remains bounded	Pass	test_integration. TestPacketStoreToPcap.test_ring_buffer_is_bounded
TC-20	Integration	Duplicate alert dispatch	Equivalent alert is published once	Pass	test_integration. TestDispatcherFanOut.test_duplicate_alert_is_published_once
TC-21	Integration	Duplicate active response request	One firewall rule is recorded for the address	Pass	test_integration. TestDispatcherFanOut.test_duplicate_block_inserted_once

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-22	Integration	Distributed high rate traffic	No single source is attributed or blockable	Pass	test_integration.TestIntervalSourceAttribution.test_distributed_traffic_is_not_attributed_or_blockable
TC-23	Unit	ATT&CK and risk mapping	Technique is mapped and risk remains within bounds	Pass	test_modules.TestMitre.test_risk_score_is_bounded
TC-24	Unit	Payload privacy limits	Control bytes are removed and excerpt stops at 64 bytes	Pass	test_modules.TestPrivacy.test_control_bytes_cannot_reach_the_log
TC-25	Unit	PCAP endian, time and snap length	Reader preserves valid capture semantics	Pass	test_modules.TestPcap.test_write_then_read_round_trip
TC-26	System	Malformed SIEM datagram	Listener rejects input and remains available	Pass	test_siem_api.malformed_datagram_case
TC-27	System	Statistics and protocol filtering	HTTP API reports live values and filtered packet rows	Pass	test_siem_api.statistics_and_filter_endpoints

ID	Level	Scenario	Expected outcome	Status	Evidence
TC-28	Training	Dataset missing required columns	Loader raises a clear validation error	Pass	test_training_and_eval missing column validation

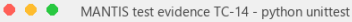
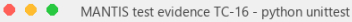
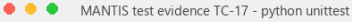
Table 5.4 reproduces the white terminal execution evidence for every case in Table 5.3. Each screenshot shows the exact unittest command, the engine or pipeline output produced during the run and the final OK verdict, captured on 20 August 2026.

Table 5.4: Terminal execution evidence for test cases TC-01 to TC-28.

ID	Executed test evidence (white terminal)
TC-01	 <pre> MANTIS test evidence TC-01 - python unittest \$ python -m unittest -v tests.test_engines.TestNaiveBayes.test_normal_http_is_safe test_normal_http_is_safe (tests.test_engines.TestNaiveBayes.test_normal_http_is_safe) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-02	 <pre> MANTIS test evidence TC-02 - python unittest \$ python -m unittest -v tests.test_engines.TestNaiveBayes.test_sqli_is_malicious test_sqli_is_malicious (tests.test_engines.TestNaiveBayes.test_sqli_is_malicious) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-03	 <pre> MANTIS test evidence TC-03 - python unittest \$ python -m unittest -v tests.test_modules.TestKNNFeatureScaling.test_flood_is_not_reported_as_a_port_scan [*] KNN Initialized with 12 training samples. test_flood_is_not_reported_as_a_port_scan (tests.test_modules.TestKNNFeatureScaling.test_flood_is_not_reported_as_a_port_scan) ... ok ----- Ran 1 test in 0.000s OK </pre>

TC-04	MANTIS test evidence TC-04 - python unittest <pre> \$ python -m unittest -v tests.test_engines.TestZScore.test_spike_triggers_high_z tests.test_modules.TestZScoreCorrections.test_sustained_flood_is_detected_throughout [ZS] 10 observations mean=10.90 std=1.14 peak=13 test_spike_triggers_high_z (tests.test_engines.TestZScore.test_spike_triggers_high_z) ... ok test_sustained_flood_is_detected_throughout (tests.test_modules.TestZScoreCorrections.test_sustained_flood_is_detected_throughout) ... ok ----- Ran 2 tests in 0.001s OK </pre>
TC-05	MANTIS test evidence TC-05 - python unittest <pre> \$ python -m unittest -v tests.test_engines.TestBloom.test_added_item_found tests.test_engines.TestBloom.test_unknown_item_not_found test_added_item_found (tests.test_engines.TestBloom.test_added_item_found) ... ok test_unknown_item_not_found (tests.test_engines.TestBloom.test_unknown_item_not_found) ... ok ----- Ran 2 tests in 0.000s OK </pre>
TC-06	MANTIS test evidence TC-06 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestParserToEngines.test_truncated_frame_does_not_raise test_truncated_frame_does_not_raise (tests.test_integration.TestParserToEngines.test_truncated_frame_does_not_raise) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-07	MANTIS test evidence TC-07 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestDispatcherFanOut.test_pii_is_redacted_before_persistence [!!!] MALICIOUS PAYLOAD! Keywords: ['UNION'] test_pii_is_redacted_before_persistence (tests.test_integration.TestDispatcherFanOut.test_pii_is_redacted_before_persistence) ... ok ----- Ran 1 test in 0.012s OK </pre>
TC-08	MANTIS test evidence TC-08 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestDispatcherFanOut.test_blocking_disabled_by_default tests.test_integration.TestDispatcherFanOut.test_blocking_when_enabled [!!!] DDoS DETECTED! Z-Score: 9.1 (PPS: 9000) [!!!] DDoS DETECTED! Z-Score: 9.1 (PPS: 9000) test_blocking_disabled_by_default (tests.test_integration.TestDispatcherFanOut.test_blocking_disabled_by_default) ... ok test_blocking_when_enabled (tests.test_integration.TestDispatcherFanOut.test_blocking_when_enabled) ... ok ----- Ran 2 tests in 0.005s OK </pre>

TC-09	<pre> ● ● ● MANTIS test evidence TC-09 - python unittest \$ python -m unittest -v tests.test_siem_api.TestSiemApi.test_json_alerts_are_ingested_with_attack_context [*] SIEM UDP Listener started on port 9999 [*] Starting MANTIS SIEM Web Dashboard on http://localhost:8080 [*] Press Ctrl+C to stop. [+] 127.0.0.1 T1190 MALICIOUS PAYLOAD! Keywords: ['UNION', 'SELECT'] [+] 127.0.0.1 T1498 DDoS DETECTED! Z-Score: 5.10 (PPS: 9000) [+] 127.0.0.1 T1046 PORT SCAN DETECTED! (Unique Ports: 210/sec) C:\Users\Anuskar\AppData\Local\Programs\Python\Python314\Lib\unittest\suite.py:166: ResourceWarning: unclosed <socket.socket fd=424, family=2, type=2, proto=0, laddr=('0.0.0.0', 55228)> setUpClass() ResourceWarning: Enable tracemalloc to get the object allocation traceback test_json_alerts_are_ingested_with_attack_context (tests.test_siem_api.TestSiemApi.test_json_alerts_are_ingested_with_attack_context) ... ok ----- Ran 1 test in 2.538s OK </pre>
TC-10	<pre> ● ● ● MANTIS test evidence TC-10 - python unittest \$ python -m unittest -v tests.test_integration.TestPacketStoreToPcap.test_buffer_exports_as_readable_pcap test_buffer_exports_as_readable_pcap (tests.test_integration.TestPacketStoreToPcap.test_buffer_exports_as_readable_pcap) ... ok ----- Ran 1 test in 0.006s OK </pre>
TC-11	<pre> ● ● ● MANTIS test evidence TC-11 - python unittest \$ python -m unittest -v tests.test_integration.TestReplayPipeline.test_every_engine_detects_its_own_attack [+] wrote C:\Users\Anuskar\AppData\Local\Temp\tmpp48cs3d2\scenario.pcap (13159 packets, opens in Wireshark) phase window expected detection ----- baseline browsing 0-14s no alert expected port scan 15-19s KNN -> Scan SYN flood 20-27s Z-Score -> DDoS web payloads 28s Naive Bayes -> Malicious known-bad IP 29s Bloom Filter -> Blacklisted [*] KNN Initialized with 12 training samples. test_every_engine_detects_its_own_attack (tests.test_integration.TestReplayPipeline.test_every_engine_detects_its_own_attack) ... ok ----- Ran 1 test in 0.112s OK </pre>
TC-12	<pre> ● ● ● MANTIS test evidence TC-12 - python unittest \$ python -m unittest -v tests.test_integration.TestBrainRoundTripAcrossEngines.test_all_five_engines_round_trip [*] KNN Initialized with 12 training samples. [*] KNN Initialized with 12 training samples. [*] Federate Learning: Z-Score restored baseline (mean=11.00 over 7 observations). [*] Federate Learning: KNN loaded 12 external signatures. [*] Federate Learning: NaiveBayes loaded 46 keyword likelihoods. [*] Federate Learning: K-Means loaded 3 external centroids. [*] Federated Learning: Bloom restored 1 indicators. test_all_five_engines_round_trip (tests.test_integration.TestBrainRoundTripAcrossEngines.test_all_five_engines_round_trip) ... ok ----- Ran 1 test in 0.001s OK </pre>

TC-13	 <pre> \$ python -m unittest -v tests.test_modules.TestKMeansNovelty.test_no_alert_during_warm_up test_no_alert_during_warm_up (tests.test_modules.TestKMeansNovelty.test_no_alert_during_warm_up) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-14	 <pre> \$ python -m unittest -v tests.test_modules.TestKMeansNovelty.test_anomalies_do_not_poison_the_model test_anomalies_do_not_poison_the_model (tests.test_modules.TestKMeansNovelty.test_anomalies_do_not_poison_the_model) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-15	 <pre> \$ python -m unittest -v tests.test_modules.TestKNNFeatureScaling.test_low_volume_stealth_scan_is_detected [*] KNN Initialized with 12 training samples. test_low_volume_stealth_scan_is_detected (tests.test_modules.TestKNNFeatureScaling.test_low_volume_stealth_scan_is_detected) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-16	 <pre> \$ python -m unittest -v tests.test_modules.TestConfig.test_bad_env_value_does_not_crash [!] ignoring bad config value for zscore_window: 'not-a-number' test_bad_env_value_does_not_crash (tests.test_modules.TestConfig.test_bad_env_value_does_not_crash) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-17	 <pre> \$ python -m unittest -v tests.test_modules.TestBloomThreatIntel.test_feed_loads_and_strips_comments tests.test_modules.TestBloomThreatIntel.test_no_false_negatives [*] threat intel: 3 indicators loaded from feed.txt (est. FP rate 0.0006%) test_feed_loads_and_strips_comments (tests.test_modules.TestBloomThreatIntel.test_feed_loads_and_strips_comments) ... ok test_no_false_negatives (tests.test_modules.TestBloomThreatIntel.test_no_false_negatives) ... ok ----- Ran 2 tests in 0.007s OK </pre>

TC-18	 MANTIS test evidence TC-18 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestParserToEngines.test_vlan_tagged_frame_is_decoded tests.test_integration.TestParserToEngines.test_icmp_frame_is_decoded test_vlan_tagged_frame_is_decoded (tests.test_integration.TestParserToEngines.test_vlan_tagged_frame_is_decoded) ... ok test_icmp_frame_is_decoded (tests.test_integration.TestParserToEngines.test_icmp_frame_is_decoded) ... ok ----- Ran 2 tests in 0.000s OK </pre>
TC-19	 MANTIS test evidence TC-19 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestPacketStoreToPcap.test_ring_buffer_is_bounded test_ring_buffer_is_bounded (tests.test_integration.TestPacketStoreToPcap.test_ring_buffer_is_bounded) ... ok ----- Ran 1 test in 0.003s OK </pre>
TC-20	 MANTIS test evidence TC-20 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestDispatcherFanOut.test_duplicate_alert_is_published_once [!!!] MALICIOUS PAYLOAD! Keywords: ['UNION'] test_duplicate_alert_is_published_once (tests.test_integration.TestDispatcherFanOut.test_duplicate_alert_is_published_once) ... ok ----- Ran 1 test in 0.003s OK </pre>
TC-21	 MANTIS test evidence TC-21 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestDispatcherFanOut.test_duplicate_block_inserted_once [!!!] DDoS DETECTED! Z-Score: 9.1 (PPS: 9000) test_duplicate_block_inserted_once (tests.test_integration.TestDispatcherFanOut.test_duplicate_block_inserted_once) ... ok ----- Ran 1 test in 0.005s OK </pre>
TC-22	 MANTIS test evidence TC-22 - python unittest <pre> \$ python -m unittest -v tests.test_integration.TestIntervalSourceAttribution.test_distributed_traffic_is_not_attributed_or_block able test_distributed_traffic_is_not_attributed_or_blockable (tests.test_integration.TestIntervalSourceAttributio n.test_distributed_traffic_is_not_attributed_or_blockable) ... ok ----- Ran 1 test in 0.000s OK </pre>

TC-23	 MANTIS test evidence TC-23 - python unittest <pre> \$ python -m unittest -v tests.test_modules.TestMitre.test_risk_score_is_bounded test_risk_score_is_bounded (tests.test_modules.TestMitre.test_risk_score_is_bounded) ... ok ----- Ran 1 test in 0.000s OK </pre>
TC-24	 MANTIS test evidence TC-24 - python unittest <pre> \$ python -m unittest -v tests.test_modules.TestPrivacy.test_control_bytes_cannot_reach_the_log tests.test_modules.TestPrivacy.test_excerpt_truncated_to_64_bytes test_control_bytes_cannot_reach_the_log (tests.test_modules.TestPrivacy.test_control_bytes_cannot_reach_the_log) ... ok test_excerpt_truncated_to_64_bytes (tests.test_modules.TestPrivacy.test_excerpt_truncated_to_64_bytes) ... ok ----- Ran 2 tests in 0.000s OK </pre>
TC-25	 MANTIS test evidence TC-25 - python unittest <pre> \$ python -m unittest -v tests.test_modules.TestPcap.test_write_then_read_round_trip test_write_then_read_round_trip (tests.test_modules.TestPcap.test_write_then_read_round_trip) ... ok ----- Ran 1 test in 0.006s OK </pre>
TC-26	 MANTIS test evidence TC-26 - python unittest <pre> \$ python -m unittest -v tests.test_siem_api.TestSiemApi.test_malformed_datagram_is_rejected_not_crashing [*] SIEM UDP Listener started on port 9999 [*] Starting MANTIS SIEM Web Dashboard on http://localhost:8080 [*] Press Ctrl+C to stop. [+] 127.0.0.1 T1190 MALICIOUS PAYLOAD! Keywords: ['UNION', 'SELECT'] [+] 127.0.0.1 T1498 DDoS DETECTED! Z-Score: 5.10 (PPS: 9000) [+] 127.0.0.1 T1046 PORT SCAN DETECTED! (Unique Ports: 210/sec) [+] 127.0.0.1 - INVALID UDP PAYLOAD (Raw Binary/Port Scan detected on SIEM Listener) C:\Users\Anuskar\AppData\Local\Programs\Python\Python314\Lib\unittest\suite.py:166: ResourceWarning: unclosed <socket.socket fd=428, family=2, type=2, proto=0, laddr=('0.0.0.0', 55231)> setUpClass() ResourceWarning: Enable tracemalloc to get the object allocation traceback test_malformed_datagram_is_rejected_not_crashing (tests.test_siem_api.TestSiemApi.test_malformed_datagram_is_rejected_not_crashing) ... ok ----- Ran 1 test in 3.339s OK </pre>

TC-27	   MANTIS test evidence TC-27 - python unittest <pre data-bbox="345 235 1370 709"> \$ python -m unittest -v tests.test_siem_api.TestSiemApi.test_stats_endpoint_reports_live_traffic tests.test_siem_api.TestSiemApi.test_protocol_filter [*] SIEM UDP Listener started on port 9999 [*] Starting MANTIS SIEM Web Dashboard on http://localhost:8080 [*] Press Ctrl+C to stop. [+] 127.0.0.1 T1190 MALICIOUS PAYLOAD! Keywords: ['UNION', 'SELECT'] [+] 127.0.0.1 T1498 DDoS DETECTED! Z-Score: 5.10 (PPS: 9000) [+] 127.0.0.1 T1046 PORT SCAN DETECTED! (Unique Ports: 210/sec) C:\Users\Anuskar\AppData\Local\Programs\Python\Python314\Lib\unittest\suite.py:166: ResourceWarning: unclosed <socket.socket fd=420, family=2, type=2, proto=0, laddr=('0.0.0.0', 55233)> setUpClass() ResourceWarning: Enable tracemalloc to get the object allocation traceback test_stats_endpoint_reports_live_traffic (tests.test_siem_api.TestSiemApi.test_stats_endpoint_reports_live_traffic) ... ok test_protocol_filter (tests.test_siem_api.TestSiemApi.test_protocol_filter) ... ok ----- Ran 2 tests in 2.547s OK </pre>
TC-28	   MANTIS test evidence TC-28 - python unittest <pre data-bbox="345 758 1370 953"> \$ python -m unittest -v tests.test_training_and_eval.TestLoader.test_missing_columns_raises test_missing_columns_raises (tests.test_training_and_eval.TestLoader.test_missing_columns_raises) ... ok ----- Ran 1 test in 0.004s OK </pre>

6. Future Scope

Technical scaling should move hot path counters (Bloom probes and Z-Score aggregation) into eBPF/XDP while retaining Python for orchestration, so throughput can rise without abandoning inspectable policy. Encrypted traffic coverage should use flow metadata (TLS ClientHello SNI, packet length sequences, inter arrival times) rather than interception. IPv6 parsing, authenticated dashboard access and TLS protected telemetry are required before any internet facing deployment. Feature additions include handshake and reverse path checks before iptables DROP, streaming quantile thresholds for heavy tailed rates, Kubernetes CNI aware response, and a blind class balanced replay corpus independent of the engine authors. Real world implementation is proposed in three phases: (1) supervised laboratory operation on owned hosts; (2) limited production on an isolated management VLAN with blocking disabled; (3) policy gated blocking only after false positive review against Snort or Suricata on the same traffic.

7. Conclusion

MANTIS met its aim by delivering a working, inspectable host detector on commodity Linux. The problem domain, edge hosts that cannot support heavy machine learning stacks or opaque platforms, was addressed with AF_PACKET/PCAP capture, five Standard Library engines, CSV/JSON evidence and optional iptables response. On 20 August 2026 all 116 automated tests passed in 4.642 seconds, tracing capture, classification, privacy, persistence, SIEM delivery and policy gated blocking.

The literature gaps were treated as design constraints rather than slogans. Halvorsen et al. (2024) identified the missing deployable edge path; MANTIS occupies that path with transparent mathematics instead of generative models. Neto et al. (2023) supplied labelled IoT traffic but not engine native features; MANTIS added explicit loaders. Hidayat, Ali and Arshad (2023) compared library classifiers without a packet pipeline; MANTIS integrated KNN with four complementary engines. Kus et al. (2022) showed that familiar splits overstate skill; this report therefore labels small domain matched scores as implementation checks and does not claim unknown attack identification. Relative to Snort, Suricata, Zeek and Wazuh, the artefact is a focused prototype, not a replacement. The aim is met within the stated Linux, plaintext and laboratory bounds.

MANTIS therefore meets the dissertation aim as a transparent research prototype rather than an enterprise platform replacement. Future work should evaluate longer controlled workloads, collect independent and prevalence balanced traffic, add IPv6 and richer protocol parsing, close all test sockets deterministically, and compare resource use and alert quality against Snort, Suricata, Zeek and Wazuh under the same workload. The central conclusion is that lightweight intrusion detection is technically credible only when algorithm assumptions, operational controls and evidence limitations remain visible to the operator.

8. References

- Cisco Talos (2024) Snort. [online] Available from: <https://www.snort.org/> [Accessed 18 August 2026].
- Crnkovic, G. (2010) Constructive research and info computational knowledge generation. In: Magnani, L., Carnielli, W. and Pizzi, C. (eds.) *Model Based Reasoning in Science and Technology*. Berlin: Springer, pp. 359 to 380.
- Halvorsen, J., Izurieta, C., Cai, H. and Gebremedhin, A. (2024) Applying generative machine learning to intrusion detection: a systematic mapping study and review. *ACM Computing Surveys*, 56(10), article 257. Available from: <https://doi.org/10.1145/3659575> [Accessed 18 August 2026].
- Hidayat, I., Ali, M.Z. and Arshad, A. (2023) Machine learning based intrusion detection system: an experimental comparison. *Journal of Computational and Cognitive Engineering*, 2(2), pp. 88 to 97. Available from: <https://doi.org/10.47852/bonviewJCCE2202270> [Accessed 18 August 2026].
- Kus, D., Wagner, E., Pennekamp, J., Wolsing, K., Fink, I.B., Dahlmanns, M., Wehrle, K. and Henze, M. (2022) A false sense of security? Revisiting the state of machine learning based industrial intrusion detection. In: *Proceedings of the 8th ACM Cyber Physical System Security Workshop (CPSS '22)*. New York: ACM. Available from: <https://doi.org/10.1145/3494107.3522773> [Accessed 18 August 2026].
- Mitzenmacher, M. and Upfal, E. (2017) *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*. 2nd ed. Cambridge: Cambridge University Press.
- Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R. and Ghorbani, A.A. (2023) CICIOT2023: a real time dataset and benchmark for large scale attacks in IoT environment. *Sensors*, 23(13), 5941. Available from: <https://doi.org/10.3390/s23135941> [Accessed 18 August 2026].
- National Institute of Standards and Technology (2007) *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94. Gaithersburg: NIST.
- OISF (2024) Suricata user guide. [online] Available from: <https://docs.suricata.io/> [Accessed 18 August 2026].

- Python Software Foundation (2026) Python 3 documentation. [online] Available from: <https://docs.python.org/3/> [Accessed 18 August 2026].
- Sculley, D. (2010) Web scale k-means clustering. In: Proceedings of the 19th International Conference on World Wide Web (WWW 2010). New York: ACM, pp. 1177 to 1178.
- Sommer, R. and Paxson, V. (2010) Outside the closed world: on using machine learning for network intrusion detection. In: IEEE Symposium on Security and Privacy. Los Alamitos: IEEE, pp. 305 to 316.
- Wazuh (2026) Wazuh documentation. [online] Available from: <https://documentation.wazuh.com/current/> [Accessed 18 August 2026].
- Zeek Project (2026) Zeek documentation. [online] Available from: <https://docs.zeek.org/> [Accessed 18 August 2026].

Appendix A: Gantt Chart

The Gantt chart below is the project schedule carried forward from the proposal and the interim report. The same sprint structure guided the final build: the remaining sprints completed engine corrections, the SIEM interface, the evaluation harness and this report.

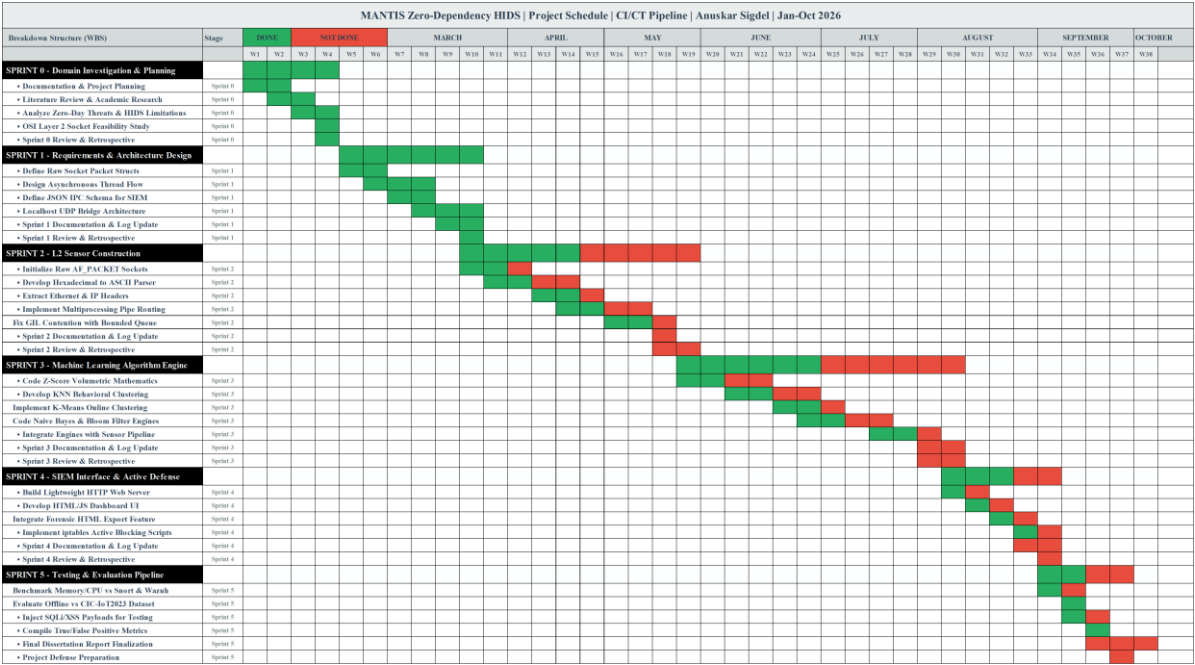


Figure A.1: MANTIS project Gantt chart.

Appendix B: Supervisor Meeting Forms

The signed project diary and log forms below record the supervision meetings held during the project, continuing the log presented in the interim report.

UNIVERSITY OF NORTHAMPTON
FACULTY OF ARTS, SCIENCE & TECHNOLOGY

PROJECT DIARY/LOG

Student: *Amrutar Sigdel*

Supervisor's signature: *Nischal Khadka*

Date: *6th March, 2026*

Last diary date: *24th February, 2026*

Objectives:

- Define technical constraints and system requirements for the MANTIS proposal.
- Analyze existing literature on applied cryptography and intrusion detection.
- Formulate a comprehensive project plan and methodology.
- Outline ethical considerations for data privacy and monitoring.

Progress:

- Requirements Engineering: Documented the constraints for a zero dependency Python build, strictly avoiding external frameworks.
- Literature Review: Summarized research on host based intrusion detection system and applied cryptography.
- Project Plan: Outlined the 4 phase methodology, from mathematical foundations to system integration.
- Ethical consideration: Addressed data privacy protocols and ethical boundaries for local host monitoring.

Supervisor's Comments:

- Identify the problem domain of the system.
- Revise and break down the project plan as per the implemented methodology.
- Identify the types of attacks implemented in your system.

Nischal Khadka
6th March 2026

ATTACHED

Figure B.1: Supervisor meeting form, project diary and log (1 of 2).

PROJECT DIARY/LOG

Student: *Anuskar Sigdel*

Supervisor's signature: *Nischal Khadka*

Date: *4th June, 2026*

Last diary date: *15th May, 2026*

Objectives:

- Draft the system design and conceptual architecture for the MANTICS framework.
- Map out of data flow and module interactions via architectural diagram.
- Initiate coding of foundational machine learning model using core Python.
- Configure a secure testing environment using virtualization.

Progress:

- system Design: Finalized the core architectural framework for the threat intelligence system.
- Diagram: Generated data flow and module architecture diagrams for the intrusion detection pipeline.
- Implementation: Coded foundation ML algorithms utilizing only the Python standard library.
- Testing: set up and testing the virtualized environment using VMware Workstation and dual boot also.

Supervisor's Comments:

- Further refinement is required in the UI/UX of the system.
- Redefine the system diagrams such as sequence diagram, system flow.
- Implement the proposed attack and identify, analyse the performance matrix.
- Prepare and enhance the test cases.

Nischal
4th June 2026

AHM/2017

Figure B.2: Supervisor meeting form, project diary and log (2 of 2).